

Министерство образования Иркутской области
Государственное бюджетное профессиональное
образовательное учреждение Иркутской области
«Иркутский авиационный техникум»
(ГБПОУИО «ИАТ»)

КП.09.02.07-3.26.231.19 ПЗ

ИНТЕРНЕТ-МАГАЗИН «ПК»

Председатель ВЦК:	_____	(А.С. Александрова)
	(подпись, дата)	
Руководитель:	_____	(Н.Р. Огородникова)
	(подпись, дата)	
Студент:	_____	(Е.А. Попов)
	(подпись, дата)	

Иркутск 2026

СОДЕРЖАНИЕ

Введение	3
1 Предпроектное исследование	4
1.1 Описание предметной области	4
1.2 Обзор инструментальных средств для разработки.....	6
2 Техническое задание	9
3 Проектирование веб-приложения	10
3.1 Архитектура веб-приложения	10
3.2 Функциональное и структурное проектирование	12
3.3 Проектирование базы данных	21
3.4 Проектирование пользовательского интерфейса	33
4 Разработка веб-приложения	45
4.1 Разработка базы данных веб-приложения	45
4.2 Разработка интерфейса веб-приложения	49
4.3 Разработка веб-приложения	55
5 Технологическая документация веб-приложения	59
5.1 Руководство гостя интернет-магазина	59
5.2 Руководство пользователя интернет-магазина	65
5.3 Руководство менеджера интернет-магазина	74
5.4 Руководство администратора интернет-магазина	78
Заключение	81
Список используемых источников.....	83
Приложение А Техническое задание	85
Приложение Б ER-модель базы данных	95

Введение

В современном мире цифровых технологий сфера онлайн-торговли демонстрирует стремительный рост и является одним из ключевых драйверов экономики. Покупка сложных технических товаров, таких как персональные компьютеры и комплектующие, через интернет стала стандартом для потребителей, что обусловлено удобством сравнения характеристик, доступностью широкого ассортимента, возможностью быстрой доставки и часто более выгодными ценами. Однако, для продавца эффективное управление таким магазином вручную сопряжено с высокими трудозатратами, рисками ошибок и низкой скоростью обработки информации. Это определяет необходимость в автоматизации бизнес-процессов интернет-магазина, что и является актуальной задачей данного курсового проекта.

Целью курсового проекта является проектирование и разработка полнофункционального интернет-магазина персональных компьютеров (ПК), предназначенного для автоматизации процессов онлайн-продажи компьютеров и комплектующих, от формирования каталога до оформления и отслеживания заказов.

Для достижения поставленной цели необходимо решить следующие задачи:

1. Провести предпроектное исследование предметной области и сформулировать требования к системе.
2. Разработать техническое задание на создание веб-приложения.
3. Спроектировать архитектуру веб-приложения, структуру базы данных и пользовательские интерфейсы.
4. Спроектировать и разработать базу данных.
5. Реализовать клиентскую и серверную часть приложения.
6. Протестировать функционал приложения и оформить сопроводительную документацию (пояснительную записку, руководство пользователя).

1 Предпроектное исследование

1.1 Описание предметной области

Предметной областью проекта является сфера электронной коммерции (B2C)– онлайн-продажа товаров категории «Персональные компьютеры и комплектующие». Разрабатываемый интернет-магазин призван автоматизировать ключевые бизнес-процессы, характерные для данной области, перенося их из физической среды в цифровую.

К основным процессам, подлежащим автоматизации, относятся:

- Управление каталогом товаров: добавление, редактирование и удаление товаров, категорий и их технических характеристик (процессоры, материнские платы, оперативная память, накопители, блоки питания, корпуса, системы охлаждения).
- Онлайн-продажи: просмотр каталога, использование интерактивного конфигуратора для подбора совместимых компонентов, управление корзиной покупок и оформление заказов.
- Управление заказами: обработка заказов менеджерами, включая подтверждение и изменение статусов («Новый», «Оплачен», «В доставке», «Выполнен»).
- Взаимодействие с клиентами: регистрация и аутентификация пользователей, ведение личного кабинета для управления профилем и просмотра истории заказов.
- Управление контентом: обновление главного товара на главной странице, публикация товаров.
- Администрирование системы: управление учётными записями, ролями пользователей, контентом главной страницы и настройками магазина.

Объекты предметной области и их характеристики:

- Товар: уникальная единица ассортимента. Характеристики: наименование, категория, цена, описание, технические спецификации (бренд, модель, сокет, тип памяти), остаток на складе, изображение.

- Категория: группировка однотипных товаров. Характеристики: название, описание, иерархическая структура (родительская категория).
- Пользователь (Покупатель): зарегистрированный клиент. Характеристики: учетные данные (email, хэш пароля), ФИО, контактная информация, адреса доставки.
- Заказ: оформленная сделка купли-продажи. Характеристики: дата и время создания, итоговая стоимость, текущий статус, способ доставки и оплаты, состав заказа.
- Позиция заказа: товарная строка в составе заказа. Характеристики: наименование товара, количество, актуальная цена на момент покупки.
- Корзина: временное хранилище выбранных товаров до момента оформления заказа. Реализована как сущность в базе данных сервера, синхронизируемая с клиентской частью через API для обеспечения сохранения состояния между различными устройствами и сессиями пользователя.

Роли пользователей в системе:

- Гость (Неавторизованный пользователь): имеет доступ к просмотру каталога, поиску товаров и добавлению их в корзину. Оформление заказа доступно только после прохождения процедуры аутентификации.
- Пользователь (Покупатель): наследует права Гостя. Дополнительно может оформлять заказы, отслеживать их статус в личном кабинете, управлять личными данными и адресами доставки.
- Менеджер: сотрудник магазина. Обладает правами Покупателя, а также доступом к панели управления для работы с каталогом товаров (CRUD-операции), обработки входящих заказов и взаимодействия с базой клиентов.
- Администратор: пользователь с максимальными привилегиями. Осуществляет полное управление системой: настройка прав доступа, управление контентом главной страницы, технический мониторинг и конфигурирование параметров приложения.

1.2 Обзор инструментальных средств для разработки

Успешная реализация проекта веб-приложения зависит от грамотного выбора технологического стека, который определяет скорость разработки, безопасность и производительность итогового продукта. Для создания интернет-магазина «ПК» был сформирован сбалансированный набор современных и широко распространённых инструментов.

Для проектирования и моделирования ключевыми инструментами стали:

1. Draw.io (diagrams.net): бесплатный онлайн-сервис для создания диаграмм. Выбран благодаря интуитивному интерфейсу, поддержке UML и ER-диаграмм, а также возможности экспорта в векторные форматы.

2. Figma: профессиональный инструмент для проектирования пользовательских интерфейсов (UI/UX). Позволяет создавать интерактивные прототипы, использовать компонентный подход и генерировать CSS-стили. Ограничения базового тарифа и зависимость от сети являются относительными недостатками.

3. MySQL Workbench: специализированная среда для проектирования и администрирования баз данных. Обеспечивает тесную интеграцию с MySQL, визуальное проектирование ER-моделей и прямое управление схемами.

Для реализации серверной части использован стек PHP + Laravel:

1. PHP 8.4: актуальная версия серверного языка с высокой производительностью, поддержкой JIT-компиляции и строгой типизацией. Преимущества: простота развёртывания, совместимость с большинством хостингов, улучшенная работа с атрибутами и типами.

2. Laravel: полнофункциональный PHP-фреймворк, реализующий паттерн MVC. Включает встроенные инструменты для маршрутизации, валидации, аутентификации (Sanctum), работы с очередями и тестирования. Обеспечивает защиту от SQL-инъекций, XSS и CSRF-атак из коробки.

3. MySQL 8.4: реляционная СУБД, выбранная за оптимальный баланс производительности и простоты интеграции с Laravel (Eloquent ORM). Поддержка транзакций гарантирует целостность данных при операциях с заказами и остатками.

Для реализации клиентской части применены:

1. React 18 + JSX: библиотека для построения компонентного интерфейса. Выбрана за использование виртуального DOM, высокую скорость рендеринга, развитую экосистему и возможность переиспользования компонентов. Управление состоянием реализуется через современные хуки (useState, useEffect, useMemo).

2. Vite: современный сборщик фронтенд-ресурсов, интегрированный в Laravel. Обеспечивает мгновенный горячий перезапуск модулей (HMR) при разработке и оптимизированную сборку для производственной среды.

3. Tailwind CSS: утилитарный CSS-фреймворк для стилизации интерфейса. Преимущества: высокая гибкость, минимальный размер итогового CSS-файла после сборки, адаптивность из коробки.

4. Axios: HTTP-клиент для выполнения асинхронных запросов к API бэкенда. Поддерживает перехватчики (interceptors) для автоматической подстановки токенов авторизации, централизованную обработку ошибок и работу с FormData.

5. Laravel Sanctum: пакет для обеспечения SPA-аутентификации. Позволяет безопасно передавать сессионные cookies между фронтендом и бэкендом, защищая API от межсайтовых запросов.

Для контроля версий исходного кода используется распределённая система Git. Она позволяет эффективно отслеживать изменения, вести разработку в изолированных функциональных ветках и при необходимости откатываться к предыдущим стабильным состояниям проекта. Хостинг репозитория и управление проектом осуществляются на платформе GitLab, которая предоставляет не только хранение кода, но и встроенные инструменты для CI/CD (непрерывной интеграции и доставки), что соответствует современным стандартам разработки.

В качестве интегрированной среды разработки (IDE) выбрана Visual Studio Code (VS Code). Редактор обусловлен своей бесплатностью, кроссплатформенностью и гибкой системой расширений. С помощью плагинов VS Code трансформируется в

мощную IDE с интеллектуальным завершением кода (IntelliSense), встроенным терминалом, отладчиком и глубокой интеграцией с Git, что делает её оптимальным инструментом для full-stack разработки.

Выбранный технологический стек обеспечивает чёткое разделение клиентской и серверной логики, высокую производительность и безопасность итогового продукта. Интеграция Laravel и React через Sanctum и Axios позволяет реализовать современную SPA-архитектуру с бесшовной навигацией и асинхронной загрузкой данных. Утилитарный подход Tailwind CSS ускоряет вёрстку и гарантирует адаптивность интерфейса на устройствах с различной шириной экрана. Использование связки Git + GitLab обеспечивает прозрачный контроль версий и возможность командной работы, а инструменты Draw.io и Figma позволяют детально проработать архитектуру и UX-дизайн до начала написания кода. Совокупность данных инструментов полностью соответствует функциональным требованиям проекта, образовательным стандартам и современным практикам веб-разработки.

2 Техническое задание

Перед разработкой было создано техническое задание, в котором указаны основные требования. Для создания технического задания был использован стандарт ГОСТ 34.602-2020. Согласно ГОСТ 34.602-2020 техническое задание должно включать следующие разделы:

1. Введение.
 - 1.1. Общие сведения.
 - 1.2. Цели и задачи.
2. Основания для разработки.
 - 2.1. Нормативные документы.
 - 2.2. Проектные документы.
3. Назначение системы.
 - 1.1. Общее описание.
 - 1.2. Преимущества и новизна.
4. Требования к системе.
 - 4.1. Функциональные требования.
 - 4.2. Технические требования.
 - 4.3. Эксплуатационные требования.
5. Требования к техническому обеспечению.
 - 5.1. Оборудование.
 - 5.2. Сетевые требования.
6. Требования к программному обеспечению.
 - 6.1. Программные компоненты.
 - 6.2. Интерфейсы.
7. Организационно-технические требования.
 - 7.1. Этапы разработки.

Полный текст технического задания представлен в Приложении А.

3 Проектирование веб-приложения

3.1 Архитектура веб-приложения

Для реализации интернет-магазина ПК была выбрана современная клиент-серверная SPA-архитектура (Single Page Application) с чётким разделением на независимые клиентскую и серверную части. Такой подход является стандартом для веб-приложений с высокой интерактивностью, таких как configurator сборных компьютеров, и обеспечивает бесшовный пользовательский опыт, масштабируемость и удобство сопровождения кода.

Архитектура приложения состоит из трёх логических уровней:

1 Уровень представления (Frontend): выполняется на стороне клиента в браузере. Реализован на базе библиотеки React 18 с использованием синтаксиса JSX. Взаимодействие с пользователем, рендеринг интерфейса, управление локальным состоянием сборки и клиентская валидация форм обеспечиваются React-хуками (useState, useEffect, useMemo). Маршрутизация реализована через React Router, а стилизация выполняется утилитарным фреймворком Tailwind CSS. Сборка и оптимизация клиентских ресурсов осуществляется инструментом Vite. Для обмена данными с сервером используется HTTP-клиент Axios, настроенный на работу с REST API. Аутентификационные запросы и получение данных о корзине выполняются асинхронно без перезагрузки страницы.

2 Уровень бизнес-логики (Backend): выполняется на веб-сервере и содержит всю серверную логику приложения. Реализован на языке PHP 8.4 с использованием фреймворка Laravel, строго следующего паттерну MVC (Model-View-Controller). Контроллеры принимают HTTP-запросы, выполняют валидацию входных данных, применяют бизнес-правила (расчёт итоговой стоимости, управление наличием товаров на складе, обработка статусов заказов) и возвращают ответы в формате JSON. Аутентификация пользователей и защита API-маршрутов реализована через пакет Laravel Sanctum, использующий безопасные сессионные cookies для связи между фронтендом и бэкендом.

3 Уровень данных (Database): представлен реляционной СУБД MySQL 8.4. Отвечает за надёжное хранение, извлечение и целостность информации. Взаимодействие с БД осуществляется через Eloquent ORM, который маппирует таблицы на PHP-модели, реализует связи между сущностями (HasMany, BelongsTo, BelongsToMany) и защищает от SQL-инъекций за счёт использования параметризованных запросов. В базе данных хранятся каталог товаров, характеристики компонентов, данные пользователей, история заказов и содержимое корзины.

Взаимодействие уровней происходит по следующему сценарию: пользователь выполняет действие в интерфейсе (например, добавляет товар в корзину или переходит на страницу товара). React-клиент отправляет асинхронный GET или POST-запрос на соответствующий эндпоинт API. Контроллер Laravel обрабатывает запрос, при необходимости обращается к модели для получения или сохранения данных в MySQL, и формирует JSON-ответ. React получает ответ и обновляет состояние компонента, мгновенно отражая изменения в интерфейсе без полной перезагрузки страницы. При оформлении заказа клиент передаёт массив выбранных товаров, сервер создаёт транзакцию, записывает заказ и его позиции в базу данных, после чего возвращает подтверждение об успешном создании.

Выбранная архитектура позволяет реализовать сложную логику обработки заказов и управления каталогом на серверной стороне с мгновенной реакцией интерфейса, а также обеспечивает лёгкое масштабирование и независимое обновление клиентской и серверной частей (рисунок 1).

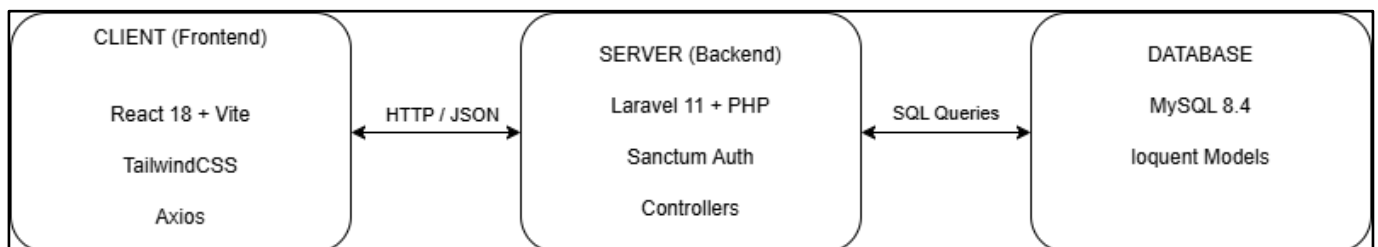


Рисунок 1 – Схема клиент-серверной архитектуры веб-приложения

3.2 Функциональное и структурное проектирование

На этапе функционального и структурного проектирования были разработаны UML-диаграммы, описывающие будущую систему с разных точек зрения. Моделирование выполнялось с использованием онлайн-сервиса Draw.io.

Цель данного этапа – зафиксировать состав функций веб-приложения интернет-магазина компьютерной техники, определить границы ответственности подсистем и распределить права доступа между категориями пользователей до начала программной реализации. Проектируемая система построена по клиент-серверной архитектуре: одностраничное веб-приложение (SPA) на React взаимодействует с серверной частью на Laravel через REST API. Аутентификация реализуется с применением Laravel Sanctum; разграничение доступа – на основе ролевой модели.

Диаграмма прецедентов (вариантов использования) описывает функциональные возможности системы и взаимодействие с ней различных типов пользователей (актеров), представлена на рисунке 2. В ходе анализа были выделены четыре основных актора:

- Гость: неавторизованный посетитель сайта. Имеет ограниченный доступ к функциям магазина.
- Покупатель: авторизованный пользователь, имеющий личный кабинет и возможность оформления заказов.
- Менеджер: сотрудник магазина, отвечающий за наполнение каталога и обработку заказов.
- Администратор: пользователь с максимальными привилегиями, управляющий настройками системы и учетными записями.

Как демонстрирует диаграмма вариантов использования, функционал системы строго регламентирован иерархией ролей. Базовый уровень доступа предоставлен актеру «Гость», который может осуществлять навигацию по каталогу, выполнять поиск товаров и формировать корзину покупок. Переход к расширенному функционалу возможен только после процедуры аутентификации, которая трансформирует роль пользователя в «Покупателя». Данная роль наследует все права гостя и добавляет возможность оформления заказов, управления личным профилем и просмотра истории транзакций.

Актер «Менеджер» расширяет права покупателя инструментами администрирования контента: управлением товарами, категориями и обработкой статусов заказов. Высший уровень привилегий принадлежит актору «Администратор», который помимо функций менеджера получает исключительное право на управление учетными записями пользователей и конфигурацию справочных таблиц.

Таким образом, диаграмма вариантов использования наглядно демонстрирует функциональные требования к системе и разграничение прав доступа для разных ролей.

Диаграмма потоков данных (Data Flow Diagram) моделирует систему с точки зрения движения информации: как данные поступают извне, где хранятся, как обрабатываются и какой результат возвращают пользователю (рисунок 3). Данная диаграмма абстрагируется от программной реализации (код, фреймворки) и фокусируется на логике бизнес-процессов.

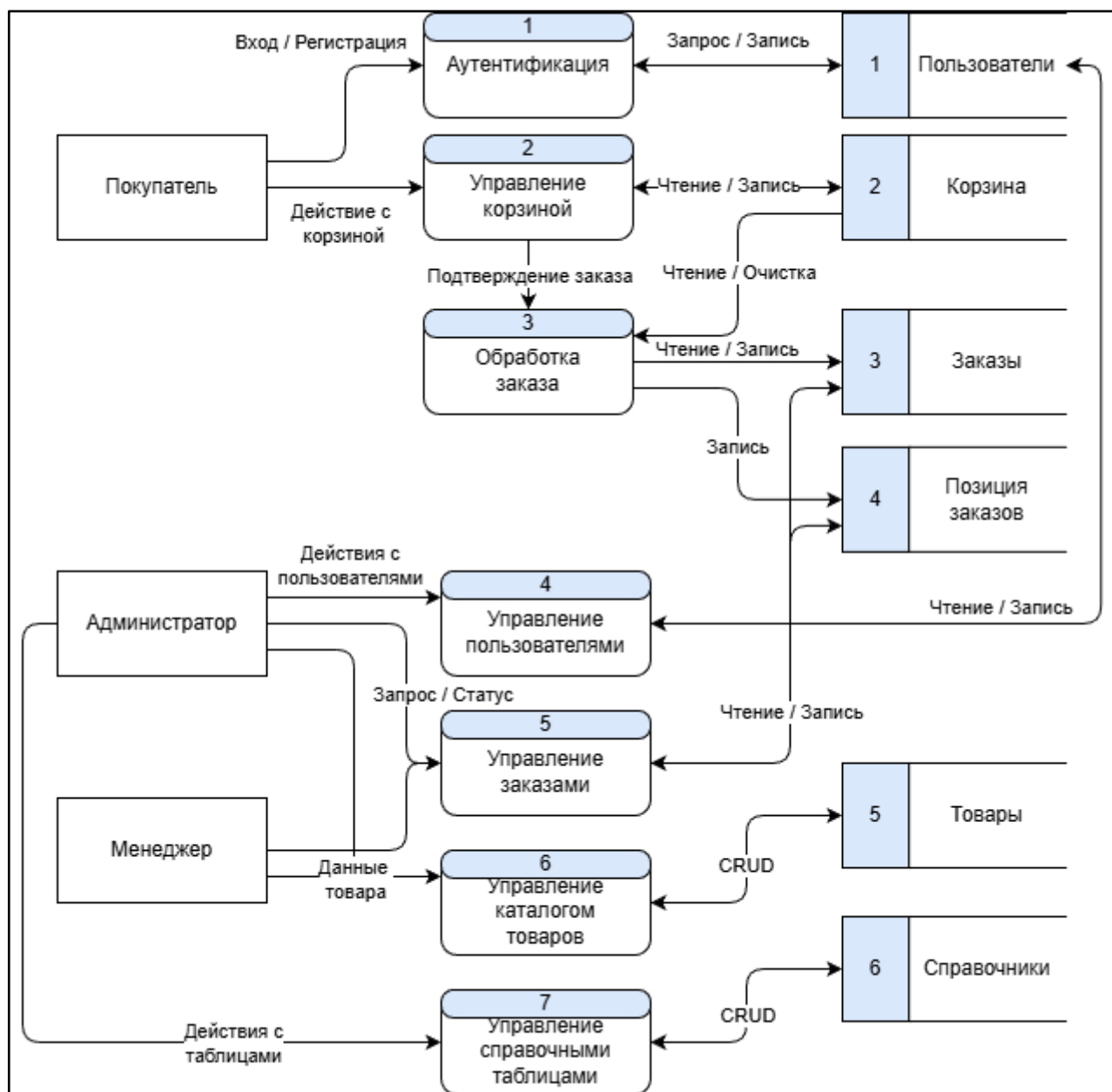


Рисунок 3 – Диаграмма потоков данных

На представленной диаграмме выделены следующие ключевые элементы:

- Внешние сущности: «Покупатель» (клиент сайта) и «Менеджер» (сотрудник магазина). Они являются источниками входных данных и получателями результатов обработки.
- Процессы: Основные операции системы, обозначенные прямоугольниками со скругленными углами.
- Хранилища данных: Базы данных, где сохраняется информация (Товары, Пользователи, Заказы, Корзины).

Описание основных потоков данных:

- Аутентификация: Покупатель отправляет в систему свои учетные данные (email и пароль). Процесс проверяет их корректность, сверяясь с хранилищем «Пользователи». В случае успеха система генерирует сессионный токен доступа и возвращает его клиенту, разрешая дальнейшую работу. При ошибке возвращается уведомление о неверных данных.

- Управление каталогом товаров: Менеджер инициирует создание или обновление товара, передавая его характеристики (название, цена, описание, остатки). Процесс выполняет валидацию данных и записывает информацию в хранилище «Товары». Обновленные данные сразу становятся доступными для отображения в каталоге на стороне клиента.

- Процесс «Управление корзиной»: Покупатель выбирает товары, что формирует поток данных «Состав корзины». Процесс временно сохраняет выбранные позиции в хранилище «Корзины», связывая их с текущим пользователем. При изменении количества или удалении товаров процесс актуализирует данные в хранилище и возвращает обновленную сумму заказа.

- Обработка заказа: Покупатель инициирует оформление заказа. Процесс извлекает данные о выбранных товарах из хранилища «Корзина» и данные пользователя. Процесс создает заказ, записывая данные в хранилища «Заказы» и «Позиции заказа», а затем инициирует очистку корзины.

- Управление заказами: Менеджер запрашивает список заказов из хранилища «Заказы». Выбрав конкретный заказ, он отправляет новый статус (например, «В доставке»). Процесс обновляет соответствующую запись в хранилище, делая информацию доступной для просмотра Покупателем в личном кабинете.

Проектирование DFD позволило четко разграничить потоки информации между клиентом и сервером, выявить необходимость промежуточного хранения данных корзины и определить строгую последовательность операций при создании заказа для обеспечения целостности данных.

Диаграмма классов (рисунок 4) является основным способом описания статической структуры системы в объектно-ориентированном программировании,

которая отображает сущности предметной области, их атрибуты (поля таблицы) и взаимосвязи (внешние ключи). Данная диаграмма послужила основой для проектирования схемы базы данных MySQL и написания моделей Eloquent в Laravel.

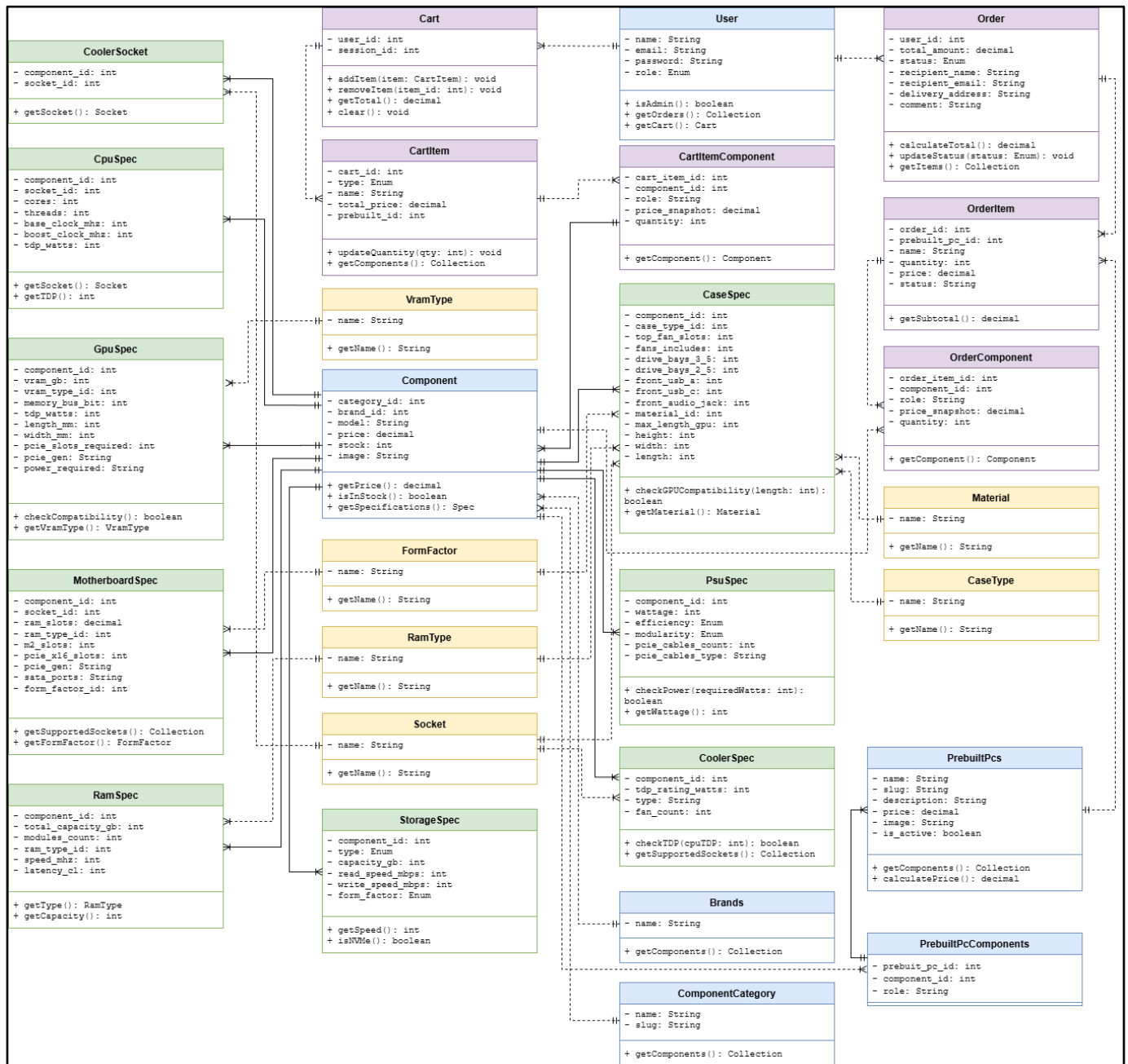


Рисунок 4 – Диаграмма классов

На диаграмме классов представлены основные сущности проекта, которые будут реализованы в виде моделей Eloquent.

- Класс User (Пользователь): содержит атрибуты: id, name, email, password, role (guest, customer, manager, admin), created_at. Имеет связь «один ко многим» с Order (пользователь может иметь много заказов) и Cart (корзина).
- Component: базовая сущность для всех комплектующих. Атрибуты: id, category_id, brand_id, model, price, stock, image. Связан с ComponentCategory (многие к одному) и Brand. В зависимости от категории, компонент имеет одну из спецификаций: CpuSpec, GpuSpec, MotherboardSpec, CaseSpec (связь один к одному).
- ComponentCategory (Категория): атрибуты: id, name, slug. Определяет тип компонента (Процессор, Видеокарта, Корпус). Связана с Component (один ко многим).
- Brand: бренды компонентов. Атрибуты: id, name, logo. Связан с Component (один ко многим).
- Класс Order (Заказ): содержит данные о заказе: дата создания, общая сумма, статус, адрес доставки. Связан с User и состоит из нескольких экземпляров класса OrderItem (композиция – элементы не существуют без заказа).
- Класс OrderItem (Позиция заказа): содержит информацию о конкретной товарной позиции в заказе: количество, цена на момент покупки (на случай, если цена товара измениться в будущем). Связывает Order и OrderComponent.
- Cart и CartItem: аналогичны структуре заказа, но используются для временного хранения выбранных товаров до оформления. Поддерживают как авторизованных пользователей (user_id), так и гостей (session_id).
- Спецификации (spec): технические характеристики для разных типов компонентов.
- Классы-справочники (Socket, FormFactor, RamType): для обеспечения строгой типизации характеристик и удобства фильтрации в конфигураторе выделены отдельные сущности для технических параметров. Они связаны с классом Component отношениями «один ко многим», позволяя группировать товары по техническим признакам (например, все процессоры с сокетом LGA1700).

Диаграмма определяет структуру базы данных, обеспечивает целостность данных и поддержку сложных запросов через ORM Eloquent.

Диаграмма последовательности (рисунок 5) детализирует взаимодействие объектов системы во времени для конкретного сценария. В качестве примера выбран ключевой процесс: «Оформление заказа пользователем», так как затрагивает все уровни архитектуры (клиент, сервер, база данных) и критически важен для бизнеса.

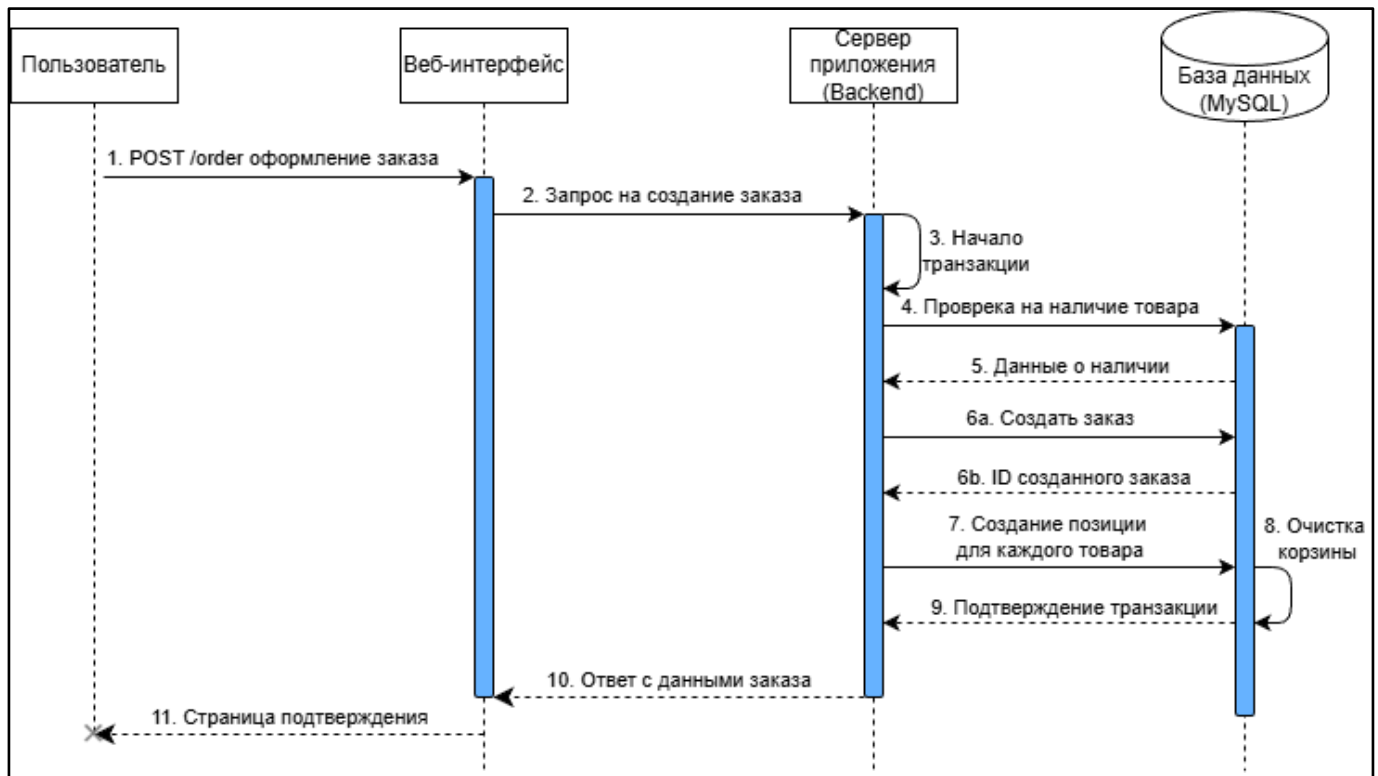


Рисунок 5 – Диаграмма последовательности

Диаграмма иллюстрирует поток событий при добавлении компонента:

- Пользователь в интерфейсе корзины нажимает кнопку «Оформить заказ».
- Клиентская валидация: React-компонент проверяет, что корзина не пуста и данные доставки заполнены.
- Запрос к API: Клиент отправляет POST-запрос на эндпоинт `/api/orders` с токеном авторизации и данными заказа (состав корзины, адрес, комментарий).
- Обработка контроллером: Laravel-контроллер `OrderController` принимает запрос. Middleware `auth:sanctum` проверяет валидность токена пользователя.
- Проверка наличия (Бизнес-логика): Контроллер запрашивает у моделей `Component` и `PrebuiltPc` актуальные остатки на складе (stock).

- Альтернативный поток: если товара нет в наличии, сервер немедленно возвращает ошибку 409 Conflict с списком недостающих товаров. Клиент отображает уведомление, процесс прерывается.

- Транзакция БД: если все товары доступны, контроллер инициирует транзакцию базы данных:

- Создается запись в таблице orders со статусом «Новый».

- Создаются связанные записи в таблице order_items (для компонентов) и prebuilt_pc_order (для готовых ПК).

- Уменьшаются остатки товаров в таблицах components и prebuilt_pcs.

- Очищается содержимое корзины пользователя в таблице cart_items.

- Ответ сервера: при успешном завершении транзакции сервер возвращает JSON с данными созданного заказа (ID, сумма, статус).

- Обновление интерфейса: React-клиент получает ответ, перенаправляет пользователя на страницу «Спасибо за заказ» и очищает локальное состояние корзины.

Диаграмма наглядно демонстрирует разделение ответственности: клиентская часть обеспечивает мгновенный UX, а серверная гарантирует безопасность и согласованность данных.

Разработанный комплекс UML-диаграмм обеспечил полное проектирование системы перед началом кодирования:

- Диаграмма вариантов использования четко разграничила права доступа между Гостем, Покупателем, Менеджером и Администратором, определив функциональные границы системы.

- DFD выявила основные потоки данных, подтвердив необходимость разделения хранилищ для товаров, заказов и временных данных корзины, а также важность серверной валидации.

- Диаграмма классов легла в основу структуры базы данных MySQL, отражая сложную нормализованную схему со спецификациями компонентов, справочниками совместимости и поддержкой готовых сборок.

– Диаграмма последовательности продемонстрировала надежный механизм обработки критических операций (заказов), гарантирующий целостность данных благодаря транзакциям и двойной проверке (клиент + сервер).

Совокупность этих моделей гарантирует, что реализованная SPA-архитектура на стеке React + Laravel полностью соответствует требованиям предметной области, обеспечивает масштабируемость каталога и безопасность сделок.

3.3 Проектирование базы данных

Проектирование базы данных является важнейшим этапом разработки веб-приложения, так как от качества структуры базы данных зависят производительность, надежность и масштабируемость всей информационной системы. Для интернет-магазина «ПК» была спроектирована реляционная база данных с использованием системы управления базами данных MySQL.

На этапе логического проектирования была разработана ER-модель (Entity-Relationship model), которая отображает сущности предметной области, их атрибуты и взаимосвязи между ними. ER-модель базы данных представлена в приложении Б ER-модель базы данных.

Разработанная ER-модель включает девять основных сущностей, которые описывают все аспекты функционирования интернет-магазина: управление пользователями, каталог товаров, процесс оформления и обработки заказов.

Таблица users (таблица 1) является центральной для системы аутентификации. Поле role реализует ролевую модель без создания отдельной таблицы, что упрощает проверку прав доступа в middleware Laravel.

Таблица 1 – Структура таблицы users (Пользователи)

Имя поля	Тип данных	Ключ	Описание
id	BIGINT	PK	Уникальный идентификатор пользователя
name	VARCHAR(255)		Название роли (guest, user, manager, admin)
email	VARCHAR(255)		Адрес электронной почты
password	VARCHAR(255)		Хэш пароля (алгоритм bcrypt)
role	ENUM		Уровень доступа (user, manager, admin)
created_at	TIMESTAMP		Дата создания записи
updated_at	TIMESTAMP		Дата обновления записи

Таблица component_categories (таблица 2) обеспечивает тематическую группировку товаров. Поле slug используется в React-роутере для динамической генерации путей каталога.

Таблица 2 – Структура таблицы component_categories (Категории компонентов)

Имя поля	Тип данных	Ключ	Описание
id	BIGINT	PK	Уникальный идентификатор категории
name	VARCHAR(100)		Полное название (например: «Процессоры», «Видеокарты»)
slug	VARCHAR(100)	UK	URL-алиас для маршрутизации
created_at	TIMESTAMP		Дата создания записи
updated_at	TIMESTAMP		Дата обновления записи

Таблица brands (таблица 3) – это справочная таблица, исключая избыточность текстовых названий в основной таблице компонентов. Связана с components отношением 1:N.

Таблица 3 – Структура таблицы brands (Бренды)

Имя поля	Тип данных	Ключ	Описание
id	BIGINT	PK	Уникальный идентификатор бренда
name	VARCHAR(100)	UK	Название производителя
created_at	TIMESTAMP		Дата создания записи
updated_at	TIMESTAMP		Дата обновления записи

Таблица components (таблица 4) – ядро каталога. Хранит общие атрибуты, применимые ко всем типам комплектующих. Специфические технические характеристики вынесены в отдельные таблицы спецификаций.

Таблица 4 – Структура таблицы components (Компоненты)

Имя поля	Тип данных	Ключ	Описание
id	BIGINT	PK	Уникальный идентификатор компонента
category_id	BIGINT	FK	Идентификатор категории
brand_id	BIGINT	FK	Идентификатор производителя
model	VARCHAR(255)		Полное маркетинговое название модели
price	DECIMAL(10,2)		Текущая розничная цена (₽)
stock	INT		Доступный остаток на складе
image	VARCHAR(255)		Относительный путь к изображению
created_at	TIMESTAMP		Дата создания записи
updated_at	TIMESTAMP		Дата обновления записи

Таблица motherboard_specs (таблица 5) содержит параметры для материнских плат.

Таблица 5 – Структура таблицы motherboard_specs (Характеристики материнской платы)

Имя поля	Тип данных	Ключ	Описание
component_id	BIGINT	PK, FK	Идентификатор материнской платы
socket_id	BIGINT	FK	Поддерживаемый сокет процессора
ram_slots	DECIMAL(10,2)		Количество слотов DIMM
ram_type_id	VARCHAR(255)		Тип поддерживаемой оперативной памяти
m2_slots	INT		Количество разъемов M.2
pcie_x16_slots	INT		Количество слотов PCIe x16
pcie_gen	VARCHAR(255)		Поколение интерфейса последовательной шины, поддерживаемый материнской платой
sata_ports	VARCHAR(255)		Количество портов SATA
form_factor_id	BIGINT	FK	Форм-фактор платы
created_at	TIMESTAMP		Дата создания записи
updated_at	TIMESTAMP		Дата обновления записи

Таблица `cpu_specs` (таблица 6) содержит параметры для центрального процессора, критически важные для алгоритма проверки совместимости (сокет, тепловыделение).

Таблица 6 – Структура таблицы `cpu_specs` (Характеристики процессора)

Имя поля	Тип данных	Ключ	Описание
<code>component_id</code>	BIGINT	PK, FK	Идентификатор процессора
<code>socket_id</code>	BIGINT	FK	Тип разъёма
<code>cores</code>	INT		Физическое количество ядер
<code>threads</code>	INT		Количество логических процессоров (потоков)
<code>base_clock_mhz</code>	INT		Базовая тактовая частота (МГц)
<code>boost_clock_mhz</code>	INT		Максимальная тактовая частота (МГц)
<code>tdp_watts</code>	INT		Тепловыделение (Вт)
<code>created_at</code>	TIMESTAMP		Дата создания записи
<code>updated_at</code>	TIMESTAMP		Дата обновления записи

Таблица `gpu_specs` (таблица 7) содержит параметры для графического процессора, критически важные для алгоритма проверки совместимости (длина, тепловыделение).

Таблица 7 – Структура таблицы `gpu_specs` (Характеристики графического процессора)

Имя поля	Тип данных	Ключ	Описание
<code>component_id</code>	BIGINT	PK, FK	Идентификатор графического процессора
<code>vram_gb</code>	INT		Объём видеопамяти (ГБ)
<code>vram_type_id</code>	BIGINT	FK	Тип видеопамяти
<code>memory_bus_bit</code>	INT		Разрядность шины памяти (бит)
<code>tdp_watts</code>	INT		Тепловыделение (Вт)
<code>length_mm</code>	INT		Физическая длина карты (мм)
<code>width_mm</code>	INT		Физическая ширина карты (мм)
<code>pcie_slots_required</code>	INT		Количество блокируемых портов PCIe
<code>pcie_gen</code>	VARCHAR(255)		Поколение интерфейса последовательной шины, используемый видеокартой
<code>power_required</code>	VARCHAR(255)		Тип используемого разъёма питания

Продолжение таблицы 7

Имя поля	Тип данных	Ключ	Описание
created_at	TIMESTAMP		Дата создания записи
updated_at	TIMESTAMP		Дата обновления записи

Таблица ram_specs (таблица 8) содержит параметры ОЗУ для расчетов конфигуратора и дополнительную информацию. Поле modules_count участвует в динамическом расчёте максимального допустимого количества комплектов относительно ram_slots материнской платы. А поле ram_type_id участвует проверке совместимости поддерживаемого типа памяти между материнской платой.

Таблица 8 – Структура таблицы ram_specs (Характеристики ОЗУ)

Имя поля	Тип данных	Ключ	Описание
component_id	BIGINT	PK, FK	Идентификатор модуля/комплекта ОЗУ
total_capacity_gb	INT		Суммарный объём памяти (ГБ)
modules_count	INT		Количество планок в комплекте
ram_type_id	BIGINT	FK	Стандарт памяти (DDR4/DDR5)
speed_mhz	INT		Эффективная тактовая частота (МГц)
latency_cl	INT		Латентность (задержка) ОЗУ (нс)
created_at	TIMESTAMP		Дата создания записи
updated_at	TIMESTAMP		Дата обновления записи

Таблица storage_specs (таблица 9) унифицирует хранение спецификаций для SSD и NVMe, позволяя конфигуратору отдельно контролировать лимиты m2_slots и sata_ports.

Таблица 9 – Структура таблицы storage_specs (Характеристики накопителей)

Имя поля	Тип данных	Ключ	Описание
component_id	BIGINT	PK, FK	Идентификатор накопителя
type	ENUM		Интерфейс подключения (nvme, sata)
capacity_gb	INT		Ёмкость хранилища (ГБ)
read_speed_mbps	INT		Линейная скорость чтения (МБ/с)
write_speed_mbps	INT		Линейная скорость записи (МБ/с)
form_factor	ENUM		Форм-фактор накопителя (m.2_2280, 2.5, 3.5)

Продолжение таблицы 9

created_at	TIMESTAMP		Дата создания записи
updated_at	TIMESTAMP		Дата обновления записи

Таблица psu_specs (таблица 10) используется для хранения спецификации блоков питания, а также содержит поле wattage, используемое для расчета безопасного запаса мощности всей системы.

Таблица 10 – Структура таблицы psu_specs (Характеристики блоков питания)

Имя поля	Тип данных	Ключ	Описание
component_id	BIGINT	PK, FK	Идентификатор блока питания
wattage	INT		Номинальная мощность (Вт)
efficiency	ENUM		Сертификат энергоэффективности
modularity	ENUM		Тип кабельной модульности
pcie_cables_count	INT		Количество кабелей питания GPU
pcie_cables_type	VARCHAR(255)		Тип кабелей питания GPU
created_at	TIMESTAMP		Дата создания записи
updated_at	TIMESTAMP		Дата обновления записи

Таблица cooler_specs (таблица 11) используется для хранения спецификации кулеров. Содержит поле tdp_rating_watts, участвующее для проверки достаточного рассеивания тепловыделения от процессора tdp_watts.

Таблица 11 – Структура таблицы cooler_specs (Характеристики кулеров)

Имя поля	Тип данных	Ключ	Описание
component_id	BIGINT	PK, FK	Идентификатор корпуса
tdp_rating_watts	INT		Рассеиваемое тепловыделение (Вт)
type	VARCHAR(50)		Конструктивный тип (башенный, AIO, боксовый)
fan_count	INT		Количество вентиляторов
created_at	TIMESTAMP		Дата создания записи
updated_at	TIMESTAMP		Дата обновления записи

Таблица case_specs (таблица 12) содержит физические ограничения и периферийные характеристики корпусов, используемые на начальных этапах проверки совместимости корпуса.

Таблица 12 – Структура таблицы case_specs (Характеристики корпусов)

Имя поля	Тип данных	Ключ	Описание
component_id	BIGINT	PK, FK	Идентификатор корпуса
case_type_id	BIGINT	FK	Габаритный класс корпуса
top_fan_slots	INT		Количество посадочных мест для вентиляторов сверху
fans_includes	INT		Штатное количество вентиляторов в комплекте
drive_bays_3_5	INT		Количество отсеков 3.5"
drive_bays_2_5	INT		Количество отсеков 2.5"
front_usb_a	INT		Порты USB Type-A на передней панели
front_usb_c	INT		Порты USB Type-C на передней панели
front_audio_jack	INT		Наличие комбинированного аудиоразъёма
material_id	BIGINT	FK	Материал боковых панелей
max_length_gpu	INT		Максимально допустимая длина видеокарты (мм)
height	INT		Высота корпуса (мм)
width	INT		Ширина корпуса (мм)
length	INT		Длина корпуса (мм)
created_at	TIMESTAMP		Дата создания записи
updated_at	TIMESTAMP		Дата обновления записи

Таблица form_factors (таблица 13) – справочник форм-факторов материнских плат. Участвует в связи M:N с корпусами через таблицу case_form_factor.

Таблица 13 – Структура таблицы form_factors (Форм-факторы)

Имя поля	Тип данных	Ключ	Описание
id	BIGINT	PK	Уникальный идентификатор стандарта
name	VARCHAR(50)	UK	Название (ATX, Micro-ATX, Mini-ITX, E-ATX)
created_at	TIMESTAMP		Дата создания записи
updated_at	TIMESTAMP		Дата обновления записи

Таблица materials (Таблица 14) – справочник конструкционных материалов. Связан с case_specs отношением 1:N для упрощения фильтрации и визуализации.

Таблица 14 – Структура таблицы materials (Материалы)

Имя поля	Тип данных	Ключ	Описание
id	BIGINT	PK	Уникальный идентификатор материала
name	VARCHAR(50)	UK	Название (Steel, Aluminum, Tempered Glass)
created_at	TIMESTAMP		Дата создания записи
updated_at	TIMESTAMP		Дата обновления записи

Таблица case_types (Таблица 15) – классификация корпусов по внутреннему объёму и габаритам.

Таблица 15 – Структура таблицы case_types (Типы корпуса)

Имя поля	Тип данных	Ключ	Описание
id	BIGINT	PK	Уникальный идентификатор типа
name	VARCHAR(50)	UK	Название (Mini-Tower, Mid-Tower, Full-Tower, SFF)
created_at	TIMESTAMP		Дата создания записи
updated_at	TIMESTAMP		Дата обновления записи

Таблица sockets (Таблица 16) – справочник процессорных разъёмов. Используется для валидации совместимости между cpu_specs, motherboard_specs и cooler_specs. Связь 1:N с таблицами спецификаций.

Таблица 16 – Структура таблицы sockets (Сокеты)

Имя поля	Тип данных	Ключ	Описание
id	BIGINT	PK	Уникальный идентификатор сокета
name	VARCHAR(50)	UK	Название разъёма (LGA1700, AM5, AM4)
created_at	TIMESTAMP		Дата создания записи
updated_at	TIMESTAMP		Дата обновления записи

Таблица ram_types (таблица 17) – справочник стандартов оперативной памяти. Обеспечивает проверку совместимости модулей ОЗУ с поддерживаемым типом материнской платы.

Таблица 17 – Структура таблицы ram_types (Типы ОЗУ)

Имя поля	Тип данных	Ключ	Описание
id	BIGINT	PK	Уникальный идентификатор типа памяти
name	VARCHAR(50)	UK	Стандарт памяти (DDR4, DDR5, LPDDR5)
created_at	TIMESTAMP		Дата создания записи
updated_at	TIMESTAMP		Дата обновления записи

Таблица vram_types (Таблица 18) – справочник для категоризации видеокарт по типу используемой видеопамяти. Связь 1:N с gru_specs.

Таблица 18 – Структура таблицы vram_types (Тип видеопамяти)

Имя поля	Тип данных	Ключ	Описание
id	BIGINT	PK	Уникальный идентификатор типа видеопамяти
name	VARCHAR(50)	UK	Тип памяти видеокарты (GDDR6, GDDR6X, GDDR5X)
created_at	TIMESTAMP		Дата создания записи
updated_at	TIMESTAMP		Дата обновления записи

Таблица case_types (таблица 19) – классификатор форм-факторов корпусов.

Таблица 19 – Структура таблицы case_types (Тип видеопамяти)

Имя поля	Тип данных	Ключ	Описание
id	BIGINT	PK	Уникальный идентификатор типа корпуса
name	VARCHAR(50)	UK	Габаритный класс (Mini-ITX, Micro-ATX)
created_at	TIMESTAMP		Дата создания записи
updated_at	TIMESTAMP		Дата обновления записи

Таблица cooler_socket (таблица 20) – промежуточная таблица, реализующая связь «многие-ко-многим» между системами охлаждения и процессорными сокетами. Позволяет одному кулеру поддерживать несколько платформ одновременно.

Таблица 20 – Структура таблицы cooler_socket (Поддерживаемые сокеты)

Имя поля	Тип данных	Ключ	Описание
component_id	BIGINT	PK, FK	ID кулера
socket_id	BIGINT	PK, FK	ID поддерживаемого сокета

Продолжение таблицы 20

created_at	TIMESTAMP		Дата создания записи
updated_at	TIMESTAMP		Дата обновления записи

Таблица `prebuilt_pcs` (таблица 21) – хранит информацию о готовых компьютерах.

Таблица 21 – Структура таблицы `prebuilt_pcs` (Готовые сборки)

Имя поля	Тип данных	Ключ	Описание
id	BIGINT	PK	Уникальный ID сборки
name	VARCHAR(255)	PK, FK	Название ПК
slug	VARCHAR(255)	UK	URL-алиас
description	TEXT		Описание
price	DECIMAL(10,2)		Стоимость
image	VARCHAR(255)		Изображение
is_active	TINYINT(1)		Доступность
created_at	TIMESTAMP		Дата создания записи
updated_at	TIMESTAMP		Дата обновления записи

Таблица `orders` (таблица 22) хранит заголовки заказов.

Таблица 22 – Структура таблицы `orders` (Заказы)

Имя поля	Тип данных	Ключ	Описание
id	BIGINT	PK	Уникальный ID заказа
user_id	BIGINT	FK	ID покупателя
total_amount	DECIMAL(10,2)		Итоговая сумма
status	ENUM		Статус (new, paid, shipped)
recipient_name	VARCHAR(255)		Имя получателя
recipient_email	VARCHAR(255)		Почта получателя
delivery_address	TEXT		Адрес доставки
comment	TEXT		Комментарий клиента
created_at	TIMESTAMP		Дата создания записи
updated_at	TIMESTAMP		Дата обновления записи

Таблица order_items (Таблица 23) содержит позиции заказа. Поддерживает как обычные компоненты, так и готовые ПК через nullable поля.

Таблица 23 – Структура таблицы order_items (Позиции заказа)

Имя поля	Тип данных	Ключ	Описание
id	BIGINT	PK	Уникальный ID позиции
order_id	BIGINT	FK	ID заказа
prebuilt_pc_id	BIGINT	FK	ID готового ПК (если есть)
name	VARCHAR(255)		Название заказа / готового ПК
quantity	INT		Количество
price	DECIMAL(10,2)		Цена на момент покупки
status	VARCHAR(255)		Статус сборки
created_at	TIMESTAMP		Дата создания записи
updated_at	TIMESTAMP		Дата обновления записи

Таблица order_components (Таблица 24) содержит все компоненты, которые присутствуют в конкретном ПК из позиции заказа.

Таблица 24 – Структура таблицы order_components (компоненты входящие в ПК)

Имя поля	Тип данных	Ключ	Описание
id	BIGINT	PK	Уникальный ID компонента
order_item_id	BIGINT	FK	ID товара из заказа
component_id	BIGINT	FK	ID компонента из таблицы components
role	VARCHAR(255)		Роль компонента
price_snapshot	DECIMAL(10,2)		Цена на момент покупки
quantity	INT		Количество
created_at	TIMESTAMP		Дата создания записи
updated_at	TIMESTAMP		Дата обновления записи

Таблица carts (Таблица 25), cart_items (Таблица 26) и cart_item_components (Таблица 27) реализуют функционал корзины покупок, аналогично структуре заказов, но для временного хранения выбора пользователя, поскольку в корзину добавлять товар может и неавторизованный пользователь.

Таблица 25 – Структура таблицы carts (Корзина)

Имя поля	Тип данных	Ключ	Описание
id	BIGINT	PK	Уникальный ID компонента
user_id	BIGINT	FK	ID пользователя
session_id	BIGINT	FK	ID сессии неавторизованного пользователя
created_at	TIMESTAMP		Дата создания записи
updated_at	TIMESTAMP		Дата обновления записи

Таблица 26 – Структура таблицы carts_items (Позиция в корзине)

Имя поля	Тип данных	Ключ	Описание
id	BIGINT	PK	Уникальный ID позиции
cart_id	BIGINT	FK	ID заказа
type	ENUM		Тип товара (prebuilt, custom)
name	VARCHAR(255)		Название товара
total_price	DECIMAL(10,2)		Полная цена
prebuilt_id	BIGINT	FK	ID готового ПК, если есть
created_at	TIMESTAMP		Дата создания записи
updated_at	TIMESTAMP		Дата обновления записи

Таблица 27 – Структура таблицы carts_item_components (Компоненты товара)

Имя поля	Тип данных	Ключ	Описание
id	BIGINT	PK	Уникальный ID компонента
cart_item_id	BIGINT	FK	ID товара из корзины
component_id	BIGINT	FK	ID компонента из таблицы components
role	VARCHAR(255)		Роль компонента
price_snapshot	DECIMAL(10,2)		Цена на момент покупки
quantity	INT		Количество
created_at	TIMESTAMP		Дата создания записи
updated_at	TIMESTAMP		Дата обновления записи

Разработанная база данных приведена к третьей нормальной форме (3НФ).

– Первая нормальная форма (1НФ): Все поля являются атомарными, отсутствуют повторяющиеся группы данных.

- Вторая нормальная форма (2НФ): База находится в 1НФ, и все неключевые атрибуты зависят от первичного ключа целиком (устранены частичные зависимости). Это достигнуто выносом спецификаций в отдельные таблицы, зависящие от `component_id`.

- Третья нормальная форма (3НФ): База находится в 2НФ, и отсутствуют транзитивные зависимости неключевых полей от первичного ключа. Например, название сокета хранится в таблице `sockets`, а в таблице `cru_specs` хранится только ссылочный `socket_id`.

Обоснованием для использования 3НФ служат:

- Отсутствие избыточности данных (дублирование названий брендов, типов памяти исключено).
- Целостность и согласованность информации (изменение названия бренда в одном месте обновляет его во всем каталоге).
- Упрощение поддержки и модификации структуры данных.
- Оптимизация использования дискового пространства.

Таким образом, спроектированная ER-модель базы данных полностью удовлетворяет функциональным требованиям интернет-магазина «ПК» и обеспечивает надежное хранение всех необходимых данных.

3.4 Проектирование пользовательского интерфейса

Для проектирования интерфейса были созданы макеты основных страниц веб-приложения в Figma. Дизайн-система построена на использовании темной темы как основной, с акцентными элементами красного цвета для кнопок целевого действия (СТА), что соответствует стилистике игрового оборудования. Шрифтовая пара обеспечивает высокую читаемость технических характеристик.

В ходе разработки были выделены следующие ключевые макеты страниц:

Макет главной страницы (рисунок 7) начинается с фиксированной навигационной панели (Header), содержащей логотип «TechLab», ссылки на основные разделы каталога («Комплектующие», «Готовые ПК»), а также иконки

профиля. Центральным элементом первого экрана является промо-блок (Hero Section) с изображением флагманской игровой сборки и призывом к действию (рисунок 6).

Ниже расположен блок «Эксклюзивные сборки», который динамически подгружает компьютеры с соответствующим тегом из базы данных. Каждая карточка в этом блоке содержит изображение, название модели и краткие характеристики. В нижней части страницы размещены категории популярных компонентов (Процессоры, Видеокарты, Материнские платы) для быстрой навигации по каталогу. Дизайн ориентирован на мгновенное привлечение внимания к новинкам и упрощение пути пользователя к покупке.

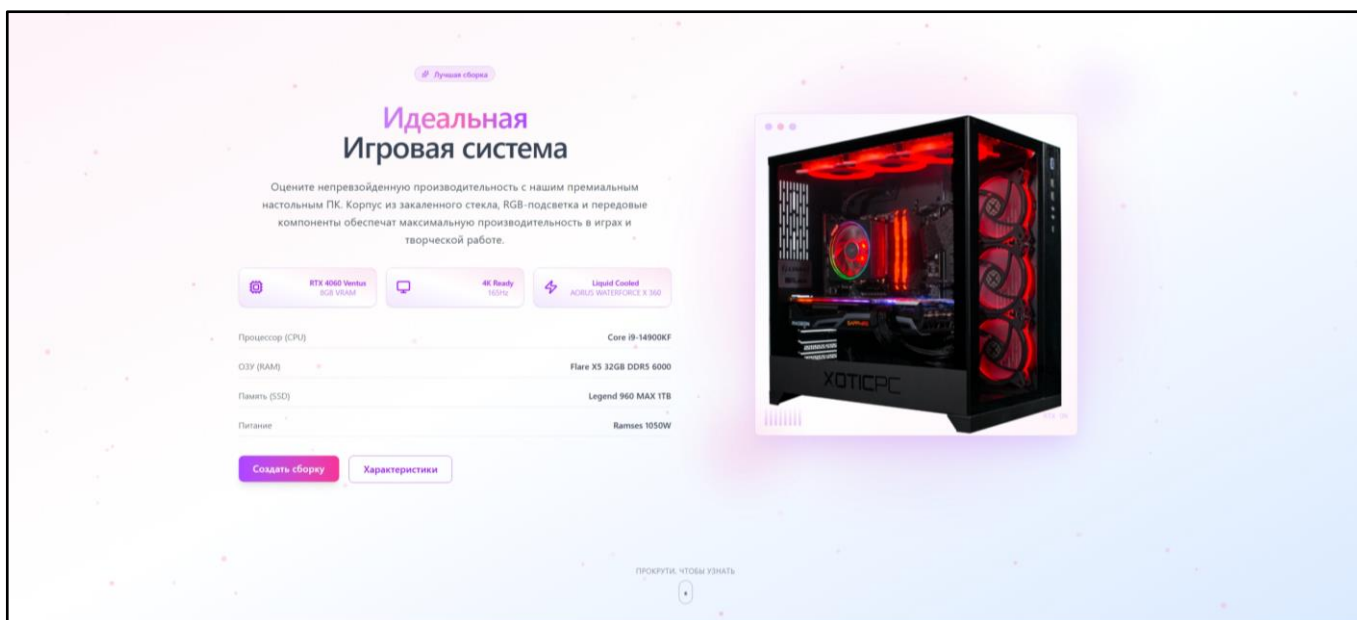


Рисунок 6 – Промо-блок

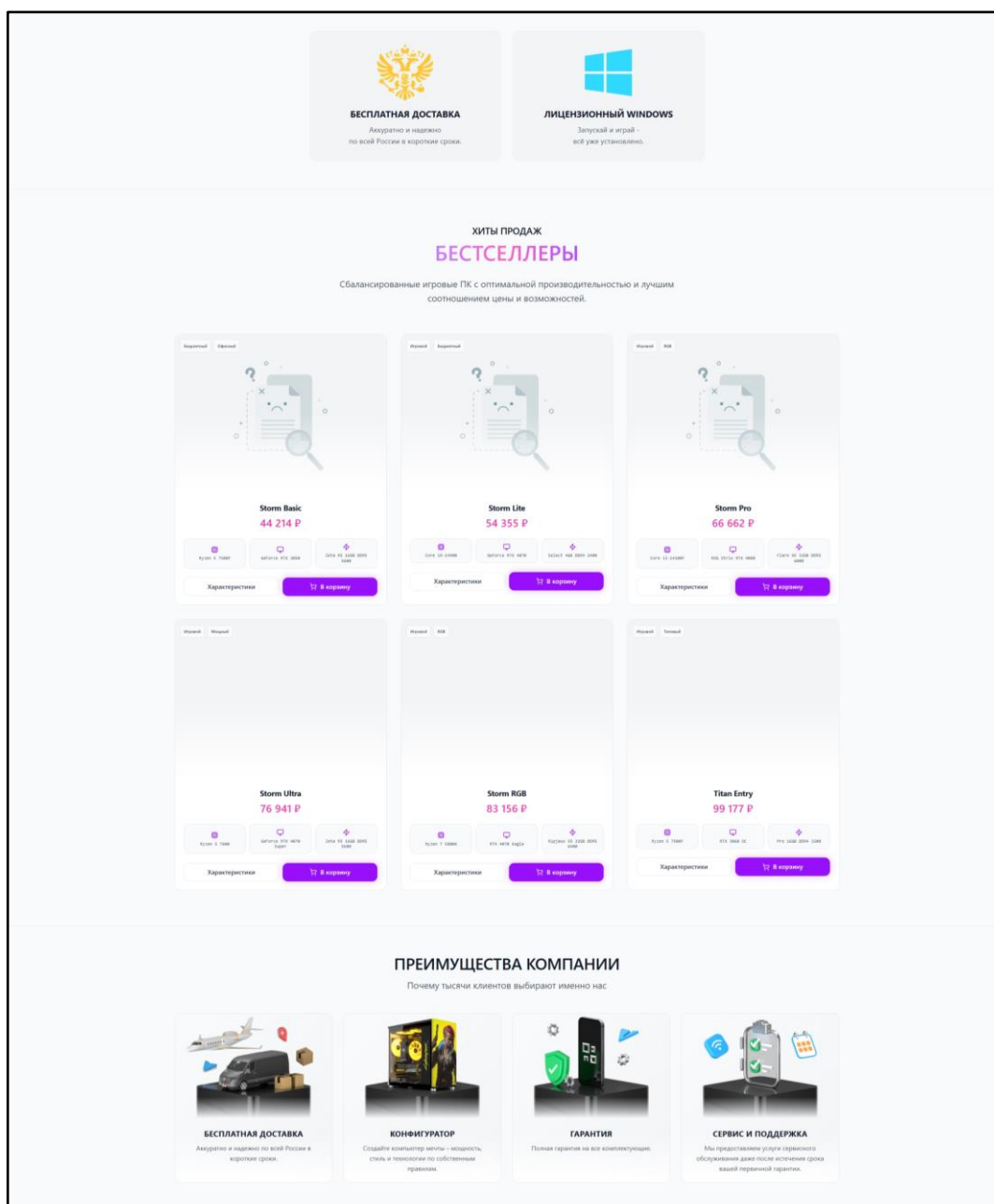


Рисунок 7 – Макет главной страницы

Макет страницы каталога компонентов (рисунок 8) предназначен для просмотра и фильтрации товаров определенной категории (например, «Процессоры» или «Видеокарты»). Страница начинается с заголовка категории и хлебных крошек (breadcrumbs) для навигации.

Основную часть занимает сетка карточек товаров. Слева (или сверху, в мобильной версии) расположен блок фильтров, позволяющий сортировать товары по бренду, цене и ключевым характеристикам (соккет, объем памяти). Карточка товара выполнена в едином стиле: содержит изображение компонента, название модели,

артикул, наличие на складе и цену. Кнопка «В корзину» позволяет быстро добавить товар без перехода на детальную страницу. Реализована пагинация для больших списков товаров.

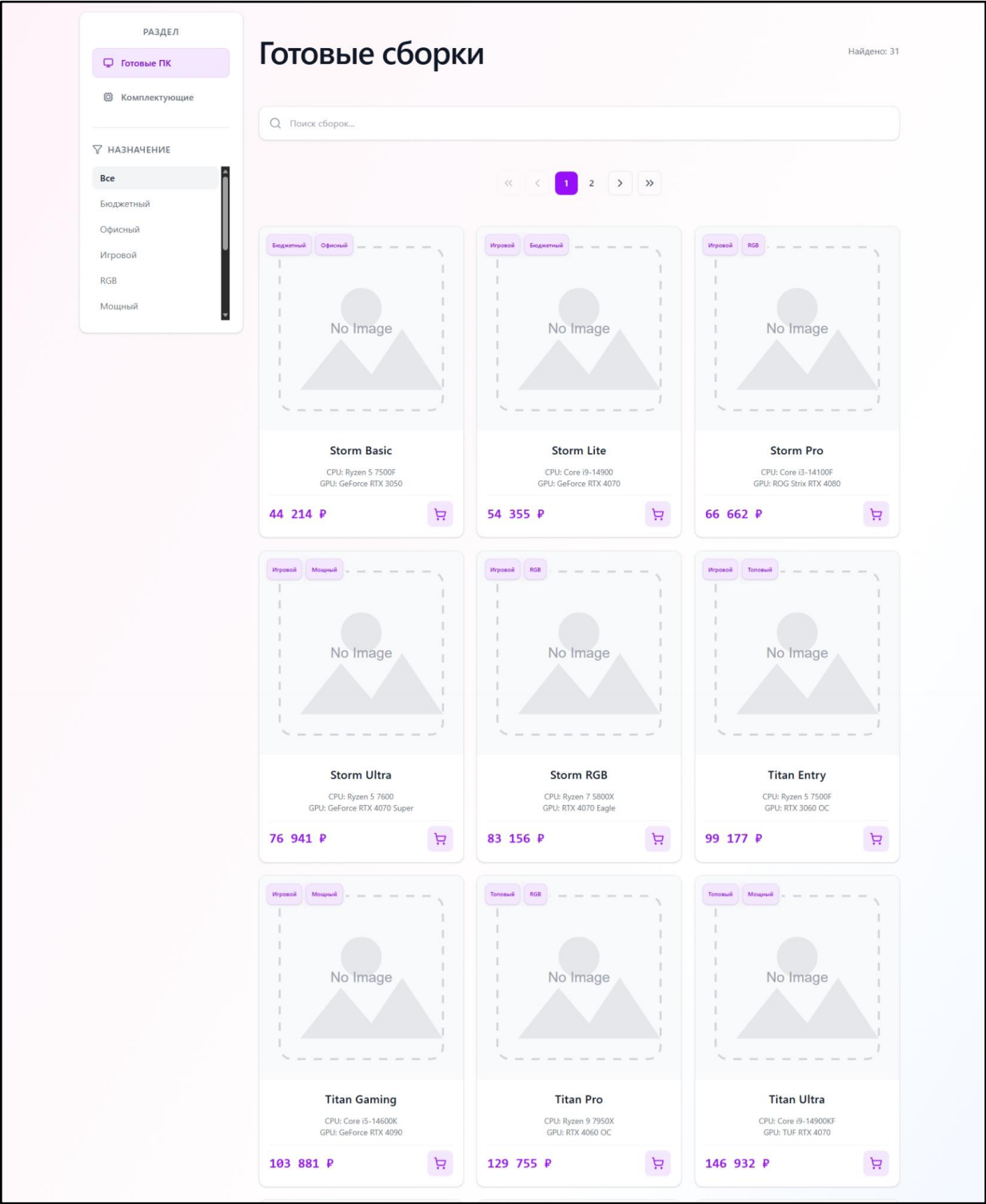


Рисунок 8 – Макет страницы каталога компонентов

Макет страницы детального просмотра товара (рисунок 9) (Component Detail Page) служит для предоставления исчерпывающей информации о конкретном компоненте. Страница разделена на две основные колонки.

В левой части расположено изображение товара. При отсутствии фотографии отображается качественный SVG-плейсхолдер, соответствующий типу компонента.

В правой части находится информационный блок: название модели, цена, статус наличия и кнопка добавления в корзину. Ниже расположен блок с подробными техническими характеристиками, сгруппированными по категориям (например, «Ядра и потоки», «Частоты», «Разъемы»). Дизайн страницы минималистичен, чтобы не отвлекать от изучения спецификаций.

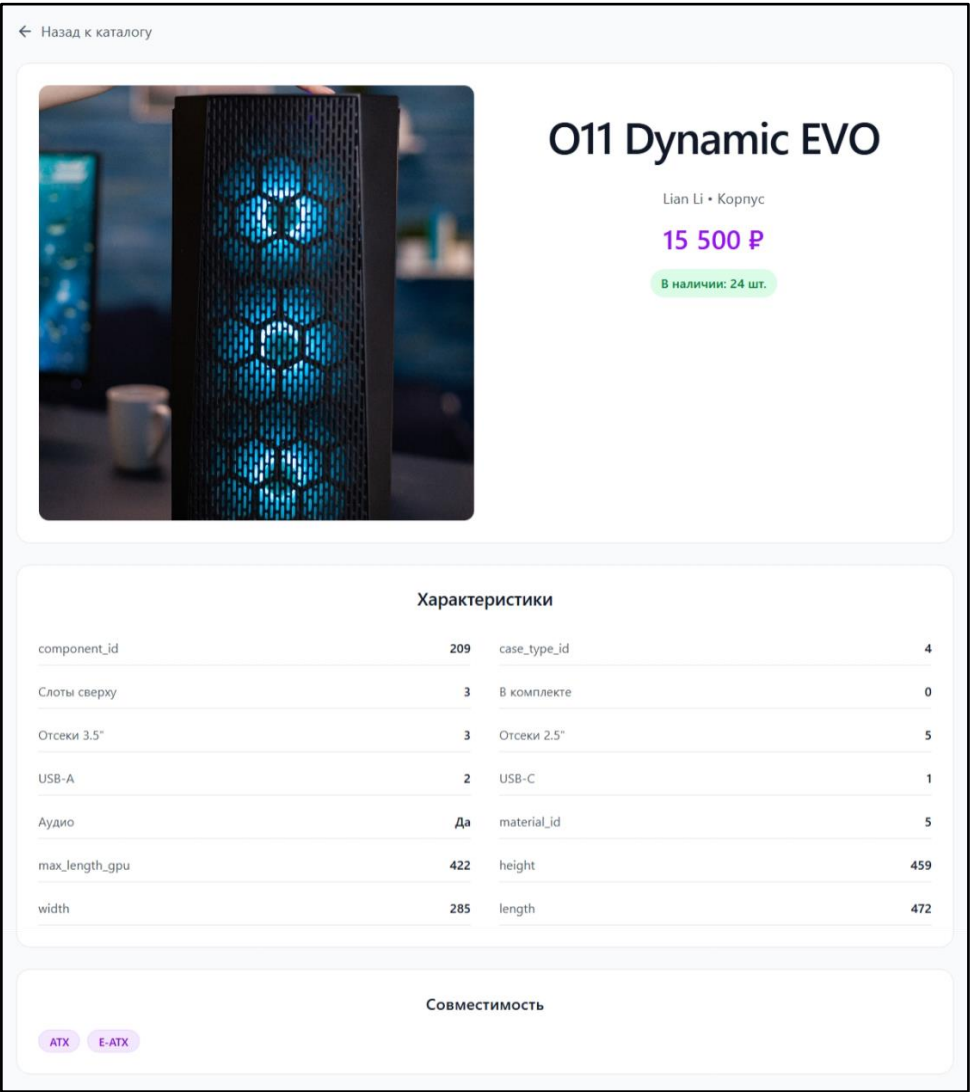


Рисунок 9 – Макет страницы детального просмотра товара

Макет страницы корзины (рисунок 10) отображает выбранные пользователем товары перед оформлением заказа. Список товаров представлен в виде таблицы или списка карточек. Для каждой позиции доступно изменение количества через счетчик, удаление товара или переход к его полному описанию.

В правой части экрана зафиксирован блок «Итоговый заказ», где в реальном времени пересчитывается сумма товаров, учитываются возможные скидки (промокоды) и отображается стоимость доставки (по умолчанию «Бесплатно» для самовывоза). Внизу расположена крупная кнопка «Оформить заказ», которая активна только при наличии товаров в корзине и авторизации пользователя. Для неавторизованных пользователей присутствует подсказка о необходимости входа для завершения покупки.

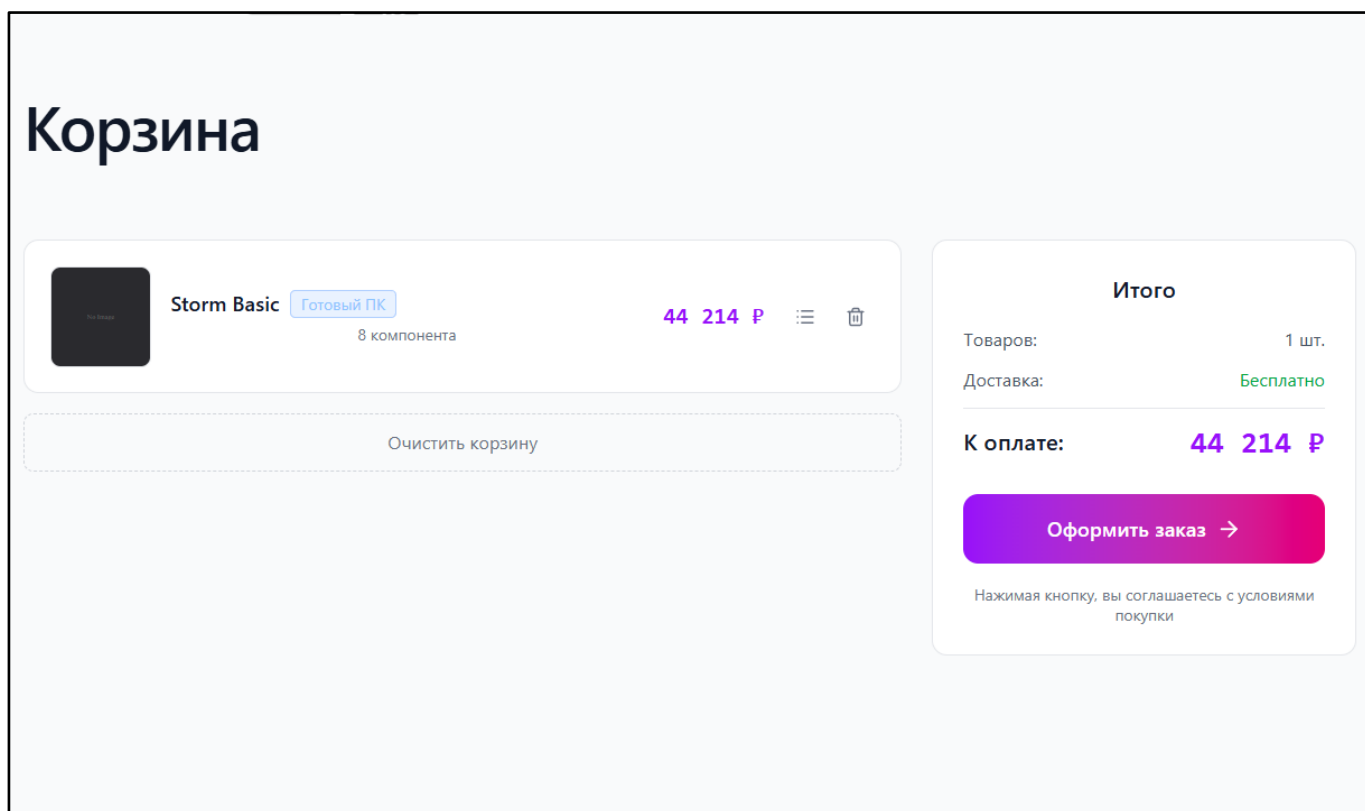


Рисунок 10 – Макет страницы корзины

Макет страницы авторизации и регистрации (рисунок 11) представляет собой минималистичный интерфейс, расположенный по центру экрана на затемненном фоне или поверх тематического изображения. Форма разделена на две вкладки: «Вход» и «Регистрация».

Поле ввода email и пароля оснащено валидацией в реальном времени: при ошибке поле подсвечивается красным, при успехе – зеленым. Кнопка «Войти» выполнена в фирменном красном цвете. Для новых пользователей предусмотрена ссылка на создание аккаунта, где дополнительно запрашиваются имя и подтверждение пароля. Внизу формы размещены ссылки на восстановление пароля и политику конфиденциальности. Дизайн исключает лишние отвлекающие элементы, фокусируя внимание пользователя на вводе данных.

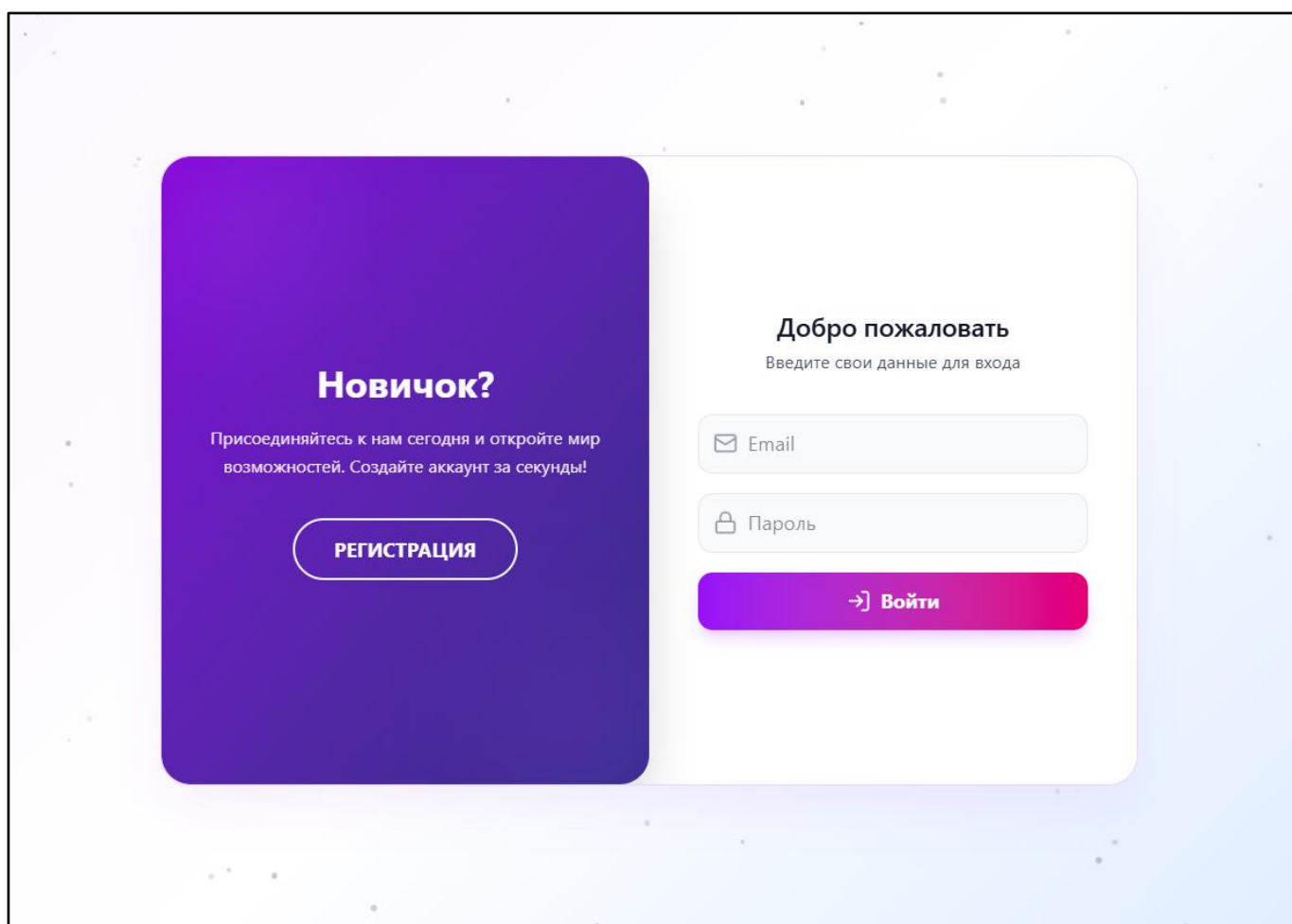


Рисунок 11 – Макет страницы авторизации

Макет страницы «О нас» (рисунок 12) рассказывает историю бренда «TechLab». Страница начинается с крупного заголовка и миссии компании: «Создаем компьютеры, которые опережают время». Ниже расположен блок с ключевыми преимуществами в виде иконок: «Гарантия 3 года», «Профессиональная сборка», «Тестирование 24 часа».

Отдельный блок посвящен команде инженеров и сборщиков с их фотографиями. В нижней части страницы размещена карта с адресом шоу-рума и контактная информация. Цветовая гамма страницы более светлая по сравнению с каталогом, чтобы создать доверительную атмосферу.

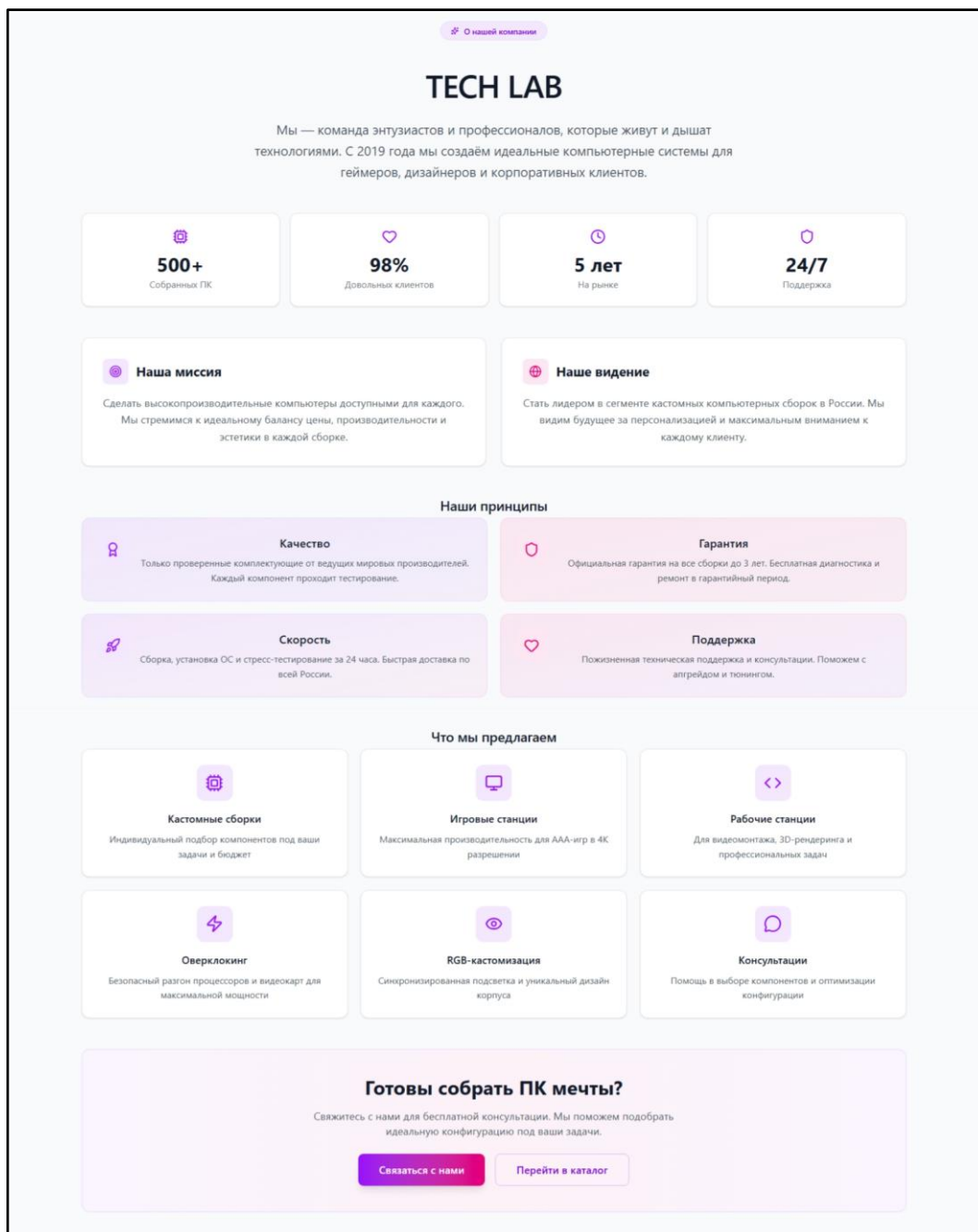


Рисунок 12 – Макет страницы «О нас»

Макет страницы профиля пользователя (рисунок 13) выполнен в строгом, но современном стиле. Страница имеет светлый фон и центрированную карточку.

В верхней части карточки расположен крупный аватар пользователя – круглая иконка с градиентным фоном (от фиолетового к розовому) и белой иконкой человека внутри. Под аватаром выводится имя пользователя и его роль («Администратор», «Менеджер» или «Пользователь») с соответствующей иконкой щита.

Основная часть страницы содержит поля «Имя» и «Email». В режиме просмотра они отображаются текстом с иконками слева. При нажатии на кнопку «Редактировать профиль» поля превращаются в инпуты с фиолетовой рамкой.

Сообщения об успехе или ошибке выводятся в цветных блоках (зеленый/красный) с полупрозрачным фоном. Кнопки действий («Сохранить», «Отмена») имеют скругленные углы и плавную анимацию нажатия.

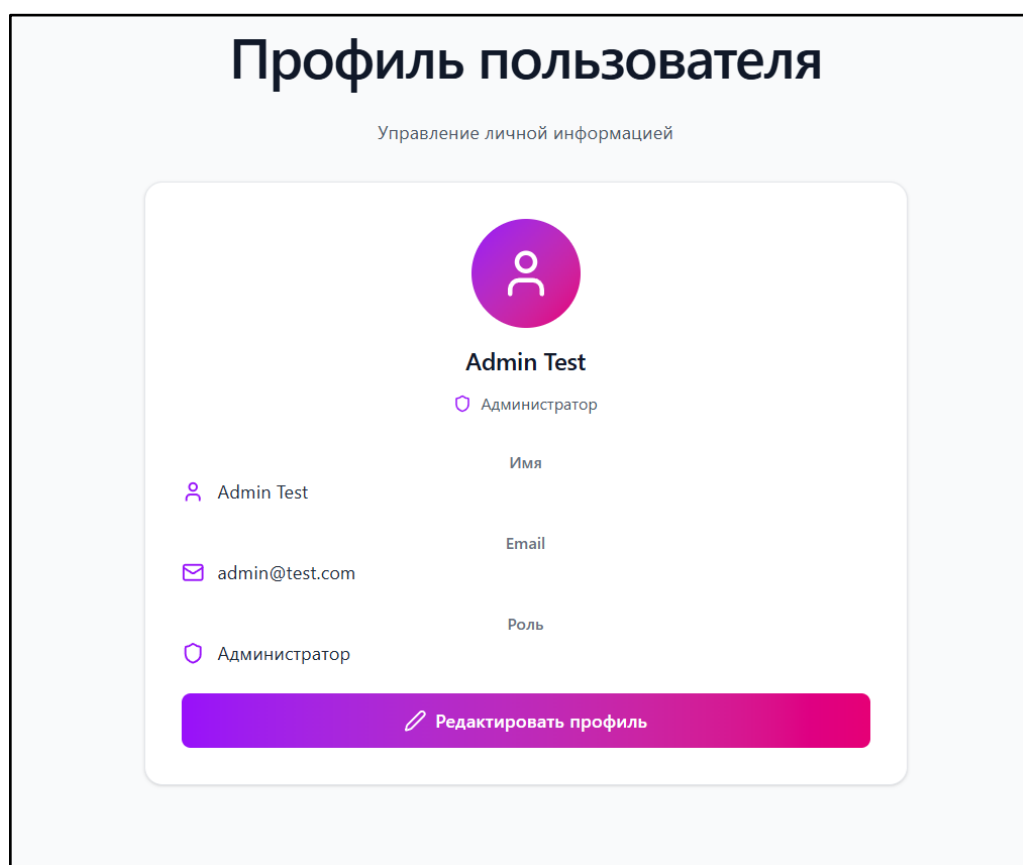


Рисунок 13 – Макет личного кабинета пользователя

Логотип (рисунок 14) является центральным элементом айдентики бренда. Он состоит из двух частей: графического знака и текстового блока.

Графический знак представляет собой стилизованное изображение дракона. Дракон символизирует мощь, энергию и премиальный характер продукции. Изображение выполнено в белом цвете (для темной темы) или черном (для светлой).

Текстовая часть содержит название «TechLab», набранное современным гротескным шрифтом с увеличенным межбуквенным интервалом.



Рисунок 14 – Логотип

Карточка товара (рисунок 15). Базовый компонент интерфейса. Имеет скругление углов rounded-2xl (радиус 16px), что делает интерфейс дружелюбным. Фон карточки светлый, без жестких теней (используется shadow-sm или его отсутствие в темной теме). Изображение товара занимает верхнюю треть. Название и цена расположены с достаточным воздухом. Кнопка действия всегда контрастна фону карточки.

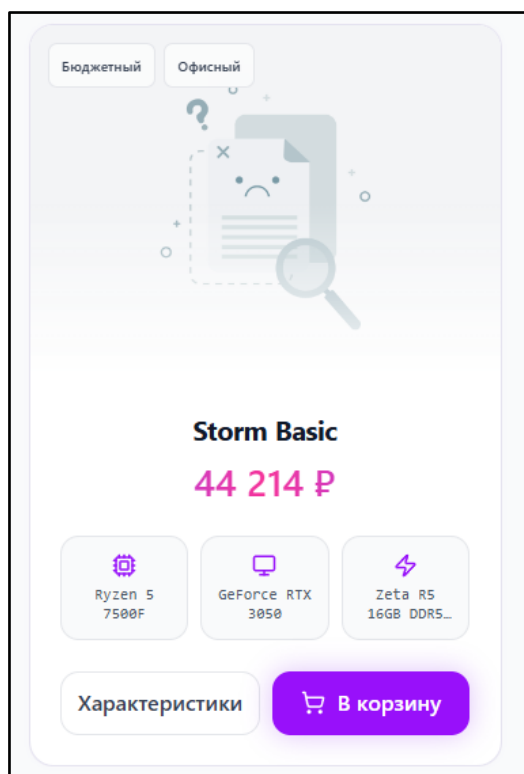


Рисунок 15 – Карточка товара

Цветовая палитра (рисунок 16):

- Фон: Глубокий черный #0f0f10 для основных поверхностей и чуть более светлый #141416 для карточек и панелей.
- Акцент: Градиент linear-gradient(from-purple-600 to-pink-600) используется для всех целевых действий (кнопки, аватары, активные ссылки). Это отличает проект от стандартных красно-черных игровых сайтов, придавая ему уникальность.
- Текст: Белый #FFFFFF для заголовков, светло-серый #9CA3AF для основного текста и второстепенной информации. Используется шрифт семейства Inter. Заголовки крупные, жирные, с плотным трекингом. Цифры в ценах и характеристиках читаемы и хорошо контрастируют с фоном.



Рисунок 16 – Логотип

Иконки (рисунок 17). Используются векторные иконки из библиотеки Lucide React (единая толщина обводки 2px). Для состояния отсутствия изображения разработан специальный белый SVG-шаблон с минималистичным контуром устройства.

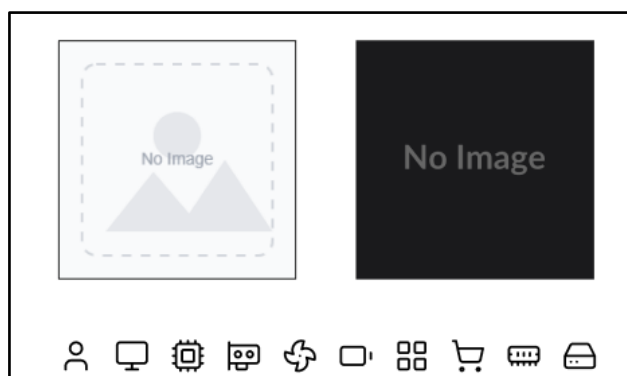


Рисунок 17 – Иконки и placeholder

Все элементы управления имеют плавные переходы (transition-all duration-300). Кнопки реагируют на наведение изменением яркости градиента. Поля ввода подсвечиваются фиолетовой рамкой при фокусе. Анимация появления контента (fade-in) реализована через библиотеку Framer Motion, что делает загрузку страниц приятной для глаза.

Таким образом, спроектированный интерфейс объединяет строгую функциональность магазина комплектующих с современной эстетикой игрового стиля, используя уникальную цветовую схему и продуманную систему компонентов.

4 Разработка веб-приложения

4.1 Разработка базы данных веб-приложения

Разработка базы данных интернет-магазина «ПК» осуществлялась на основе детальной ER-модели, спроектированной на этапе анализа требований (раздел 3.3). В качестве системы управления базами данных выбрана MySQL 8.4 – надежная реляционная СУБД, обеспечивающая целостность транзакций, поддержку сложных связей и высокую производительность при работе с большими объемами каталожных данных. Выбор обусловлен полной совместимостью со стеком PHP/Laravel и наличием мощного инструмента ORM Eloquent для абстракции работы с данными.

Физическое создание структуры базы данных выполнено с помощью механизма миграций (Migrations) фреймворка Laravel. Миграции представляют собой версионные файлы классов, описывающие схему БД в виде кода, что позволяет автоматически разворачивать структуру на любой среде разработки или производства командой `php artisan migrate`. Каждая миграция определяет поля, типы данных, индексы, первичные и внешние ключи, а также добавляет стандартные временные метки `created_at` и `updated_at` для аудита изменений.

Структура базы данных разделена на логические группы таблиц:

- 1 Таблицы пользователей и доступа: таблица `users` хранит учетные записи клиентов и сотрудников. Содержит автоинкрементный идентификатор `id`, уникальные поля `email` и `name`, хешированный пароль `password` (алгоритм `bcrypt`), а также поле `role` типа `ENUM` ('user', 'manager', 'admin') для реализации ролевой модели. Это позволяет `middleware` приложения динамически разграничивать права доступа к административным функциям (рисунок 18).

```
Schema::create('users', function (Blueprint $table) {
    $table->id();
    $table->string('name');
    $table->string('email')->unique();
    $table->enum('role', ['user', 'manager', 'admin'])->default('user');
    $table->timestamp('email_verified_at')->nullable();
    $table->string('password');
    $table->rememberToken();
    $table->timestamps();
});
```

Рисунок 18 – Миграция таблицы пользователей

Справочные таблицы и классификаторы: для обеспечения нормализации данных и гибкой фильтрации создана группа справочников: brands (бренды), component_categories (категории компонентов), sockets (типы процессорных разъемов), ram_types (стандарты памяти), form_factors (форм-факторы плат). Эти таблицы содержат поля name и уникальный slug для ЧПУ-маршрутизации. Вынесение общих характеристик в отдельные сущности исключает дублирование текстовых данных и упрощает логику проверки совместимости компонентов (Рисунок 19).

Таблица компонентов и спецификаций. Центральной сущностью каталога является таблица components. Она хранит общие атрибуты всех товаров: model (название), price (цена), stock (остаток на складе), image (путь к файлу) и внешние ключи category_id и brand_id. Поскольку технические характеристики сильно различаются для разных типов устройств (например, у процессора важен сокет, а у видеокарты – объем видеопамяти), реализована архитектура с вынесенными спецификациями. Для каждой категории компонентов создана отдельная таблица спецификаций: cpu_specs, gpu_specs, motherboard_specs, ram_specs, storage_specs, psu_specs, cooler_specs, case_specs. Каждая из этих таблиц связана с components отношением «один-к-одному» через первичный ключ component_id, который одновременно является внешним ключом. Такой подход позволяет хранить только релевантные поля для конкретного типа устройства, не засоряя основную таблицу NULL-значениями (рисунок 20).

```

public function up(): void
{
    Schema::create('components', function (Blueprint $table) {
        $table->id();
        $table->foreignId('category_id')->constrained('component_categories')->cascadeOnDelete();
        $table->foreignId('brand_id')->constrained('brands')->cascadeOnDelete();
        $table->string('model');
        $table->decimal('price', 10, 2);
        $table->integer('stock')->default(0);
        $table->string('image')->nullable();
        $table->timestamps();
    });
}

```

Рисунок 19 – Миграция таблицы компонентов

```

public function up(): void
{
    Schema::create('cpu_specs', function (Blueprint $table) {
        $table->foreignId('component_id')->primary()->constrained('components')->cascadeOnDelete();
        // $table->string('socket');
        $table->foreignId('socket_id')->constrained('sockets')->cascadeOnDelete();
        $table->integer('cores');
        $table->integer('threads');
        $table->decimal('base_clock_mhz', 5, 1);
        $table->decimal('boost_clock_mhz', 5, 1)->nullable();
        $table->integer('tdp_watts');
        $table->timestamps();
    });
}

```

Рисунок 20 – Миграция таблицы спецификаций CPU

Особое внимание уделено связям «многие-ко-многим». Например, таблица `cooler_socket` реализует поддержку кулерами нескольких типов сокетов одновременно, что критически важно для алгоритма подбора совместимого охлаждения.

Таблицы готовых сборок: для продажи предварительно собранных компьютеров предусмотрена таблица `prebuilt_pcs`. Она содержит информацию о названии сборки, итоговой цене, описании конфигурации и флаге `is_exclusive` для выделения премиальных моделей на главной странице. Связь с конкретными компонентами может реализовываться как через JSON-поле конфигурации, так и через промежуточные таблицы, в зависимости от требований к детализации учета остатков.

Хранение изображений: файлы изображений товаров и сборок физически хранятся в защищенной директории сервера `storage/app/public/`. Публичный доступ к ним организован через символическую ссылку `public/storage`, создаваемую командой `php artisan storage:link`. В базе данных (поля `image` в таблицах `components` и `prebuilt_pcs`) сохраняется только относительный путь к файлу (например, `images/prebuilt/pc_31.png`). Полные URL-адреса формируются динамически на стороне клиента с использованием базового адреса хранилища, что позволяет легко переносить приложение между доменами без обновления записей в БД.

Таблицы заказов и корзины: таблица `orders` фиксирует оформленные заказы. Содержит `user_id`, общую сумму `total_amount`, статус заказа `status` (ENUM: `'new'`, `'paid'`, `'shipping'`, `'completed'`, `'cancelled'`), адрес доставки и комментарий. Важно, что данные о составе заказа не хранятся ссылкой на текущие товары, а фиксируются на момент покупки.

Детализация заказа хранится в таблице `order_items`. Она реализует связь «один-ко-многим» с заказом и содержит `product_id` (или `prebuilt_pc_id`), `quantity` и `price` — цену единицы товара на момент совершения сделки. Это гарантирует неизменность финансовой истории даже при последующем изменении цен в каталоге.

Таблица `carts` служит временным хранилищем выбранных товаров для авторизованных пользователей, позволяя синхронизировать корзину между различными устройствами до момента оформления заказа.

Реализация связей через Eloquent ORM: взаимодействие с базой данных в коде приложения построено на использовании ORM Eloquent. В моделях Laravel определены методы отношений:

- `belongsTo()`: компонент принадлежит категории, заказ принадлежит пользователю.
- `hasMany()`: у пользователя много заказов, у категории много компонентов.
- `hasOne()`: у компонента есть одна запись спецификации (например, `cpu_specs`).

– belongsToMany(): реализовано для связей многие-ко-многим (кулеры и сокет).

Такой подход позволяет работать с связанными данными объектно-ориентированным способом (например, `$cpu -> socket -> name`), избавляя разработчика от написания сложных ручных SQL-запросов с оператором JOIN.

Нормализация базы данных: спроектированная схема приведена к третьей нормальной форме (3НФ):

1 Все неключевые атрибуты полностью зависят от первичного ключа таблицы.

2 Отсутствуют транзитивные зависимости (свойства зависят только от ключа, а не от других неключевых полей).

3 Справочные данные и спецификации вынесены в отдельные таблицы, что устраняет избыточность, предотвращает аномалии обновления и обеспечивает масштабируемость каталога при добавлении новых типов комплектующих.

Таким образом, реализованная структура базы данных полностью удовлетворяет функциональным требованиям проекта, обеспечивая надежное хранение информации о товарах, эффективную работу алгоритмов совместимости и целостность данных при обработке заказов.

4.2 Разработка интерфейса веб-приложения

Клиентская часть интернет-магазина «ПК» разработана с использованием библиотеки React 18 с синтаксисом JSX и функциональными компонентами. Сборка клиентских ресурсов осуществляется посредством современного инструментария Vite, интегрированного в экосистему Laravel. В отличие от монолитных решений с использованием Inertia.js, в проекте применена архитектура классического SPA (Single Page Application): взаимодействие с серверной частью реализовано через REST API с использованием библиотеки Axios для выполнения асинхронных HTTP-запросов, а навигация между страницами управляется библиотекой React Router DOM. Маршрутизация включает защиту приватных путей (личный кабинет, админ-

панель) через компонент `<ProtectedRoute>`, проверяющий наличие валидного токена аутентификации и роль пользователя.

Архитектура интерфейса построена по компонентному подходу. Каждый элемент UI представляет собой независимый React-компонент, принимающий данные через props и управляющий внутренним состоянием с помощью хуков `useState`, `useEffect`. Компоненты размещаются в директории `resources/js/Components/` и `resources/js/Pages/`.

Для стилизации интерфейса и обеспечения полной адаптивности под различные типы устройств использован утилитарный CSS-фреймворк Tailwind CSS. Дизайн-система приложения построена на основе темной темы (dark mode), где основной фон задается цветом `#0f0f10`, а карточки элементов – `#141416`. Акцентные действия выделены градиентами от фиолетового к розовому (`from-purple-600 to-pink-600`), что соответствует брендбуку компании. Анимация переходов и микровзаимодействий реализована с помощью библиотеки Framer Motion.

Архитектура интерфейса построена по модульному принципу. Корневой компонент `App.jsx` формирует глобальный лейаут, включающий фиксированную навигационную панель (`Navigation.jsx`) с логотипом, меню категорий и иконками профиля/корзины, а также подвал сайта (`Footer.jsx`). Основной контент динамически рендерится в зависимости от текущего маршрута.

Публичная часть приложения включает следующие ключевые страницы и компоненты:

- Главная страница (`HomePage.jsx`): содержит промо-блок (`HeroSection`) с призывом к действию и сетку эксклюзивных готовых сборок. Компонент `PremiumPCHero` реализует логику отображения сборок с тегом «Эксклюзив», включая адаптивную загрузку изображений: до момента полной загрузки отображается SVG-плейсхолдер, который плавно заменяется фотографией ПК или корпуса.

- Каталог компонентов (`ComponentListPage.jsx`): реализует фильтрацию и сортировку товаров. Сетка товаров адаптируется под размер экрана с использованием классов Tailwind `grid-cols-1 md:grid-cols-2 lg:grid-cols-4`. Карточка товара

(ComponentCard.jsx) отображает изображение, название, бренд, цену и индикатор наличия.

- Детальная страница товара (ComponentDetailPage.jsx): предоставляет исчерпывающую информацию о компоненте. Особенностью является динамическое формирование блока характеристик на основе типа компонента (процессор, видеокарта, ОЗУ). Если изображение отсутствует, автоматически подгружается тематическая иконка или фотография из связанной таблицы спецификаций.

- Страница конфигуратора и корзины: позволяет управлять выбранными товарами. Счетчик корзины в шапке сайта обновляется реактивно при добавлении товаров без перезагрузки страницы.

Административная панель (AdminDashboard, AdminComponents, AdminOrders) предоставляет инструменты для управления контентом и заказами. Доступ к данным разделам строго регламентирован: пользователи с ролью guest или user перенаправляются на главную страницу. Ключевой особенностью админки является управление сложной структурой товаров: при создании или редактировании компонента форма динамически меняет свой вид в зависимости от выбранной категории. Например, при добавлении процессора появляются поля для выбора сокета и частоты, а при добавлении видеокарты – поля для длины и типа питания. Это реализуется через условный рендеринг полей формы на основе состояния React.

Для реализации описанной архитектуры были использованы следующие программные решения и подходы:

- 1 Маршрутизация и защита данных. Навигация реализована через react-router-dom. Для разграничения прав доступа создан оберточный компонент ProtectedRoute, который при монтировании проверяет состояние контекста аутентификации (AuthContext). Если пользователь не авторизован или его роль недостаточна (например, доступ к /admin имеет только manager и admin), происходит редирект на страницу входа (/login). Конфигурация роутера с защищенными путями представлена на рисунке 21.

```

{/* Защищённые админ-роуты */}
<Route
  path="/admin/*"
  element={
    <ProtectedRoute allowedRoles={['admin', 'manager']}>
      <AdminPanel />
    </ProtectedRoute>
  }
/>
<Route path="/admin/components/:id/edit" element={<ComponentEdit />} />
<Route path="/admin/components/create" element={<ComponentCreate />} />

{/* Роуты для готовых ПК */}
<Route path="/admin/prebuilt-pcs/create" element={<ProtectedRoute allowedRoles={['admin', 'manager']}>} />
<Route path="/admin/prebuilt-pcs/:id/edit" element={<ProtectedRoute allowedRoles={['admin', 'manager']}>} />

{/* Роуты для заказов (админка) */}
<Route path="/admin/orders/:id" element={<ProtectedRoute allowedRoles={['admin', 'manager']}><AdminOrd

```

Рисунок 21 – Фрагмент конфигурации маршрутизатора с защитой ролей

2 Адаптивная верстка карточки товара. Компонент PrebuiltPcCard (для готовых сборок) использует сетку Tailwind CSS для корректного отображения на мобильных и десктопных устройствах. Классы w-full, aspect-video и object-contain гарантируют, что изображение компьютера не будет искажено. Блок с характеристиками использует условный рендеринг: если массив компонентов пуст, отображается заглушка. Градиентная кнопка «В корзину» реализована через классы bg-gradient-to-r from-purple-600 to-pink-600, обеспечивая визуальную консистентность с дизайном системы. Фрагмент кода компонента представлен на рисунке 22.

```

{ /* Контент */ }
<div className="p-5 space-y-5">
  { /* Заголовок и цена */ }
  <div className="space-y-1">
    <h3 className="text-lg font-bold text-gray-900 dark:text-white tracking-tight line-clamp-1">{pc.name}
    <p className="text-2xl font-semibold bg-gradient-to-r from-purple-600 via-pink-500 to-purple-600 dark:from-purple-500 dark:via-pink-400 dark:to-purple-500 text-transparent bg-clip-text">
      {Number(pc.price).toLocaleString('ru-RU')} Р
    </p>
  </div>

  { /* Быстрые характеристики (чипы) */ }
  <div className="grid grid-cols-3 gap-2">
    {[ 'cpu', 'gpu', 'ram' ].map(role => {
      const data = components[role];
      const { icon: Icon } = ROLE_CONFIG[role];
      return (
        <div key={role} className="flex flex-col items-center justify-center p-2.5 rounded-xl
          bg-gray-50 dark:bg-white/[0.03] border border-gray-200 dark:border-white/[0.03] shadow">
          <Icon className="w-4 h-4 text-purple-600 dark:text-purple-400 mb-1.5" />
          <span className="text-[11px] text-gray-600 dark:text-gray-400 leading-tight line-clamp-2 font-sans">
            {data?.model || '-'}
          </span>
        </div>
      );
    })}
  </div>

```

Рисунок 22 – Фрагмент верстки карточки готовой сборки

3 Динамическая форма редактирования компонентов. В административной панели (AdminComponentForm) реализована логика подгрузки полей спецификаций. При выборе категории компонента (например, «Материнская плата») система через API запрашивает список доступных сокетов и форм-факторов, заполняя соответствующие выпадающие списки (<select>). Поля ввода валидируются на стороне клиента перед отправкой данных на сервер через Axios методом POST или PUT. Обработка ошибок валидации выводит сообщения под соответствующими полями формы, подсвечивая их красной рамкой (border-red-500). Фрагмент шаблона формы приведен на рисунке 23.

```

{/* GPU */}
{slug === 'gpu' && <div className="grid grid-cols-2 md:grid-cols-3 gap-4">
  <Field label="VRAM (GB) *" type="number" val={spec.vram_gb} set={v => setSpec({ ...spec, vram_gb: +v })} error={specErrors.vram_gb}>
    <Field label="Тип VRAM *" error={specErrors.vram_type_id} onChange={() => setSpecErrors(p => ({...p, vram_type_id: +e.target.value}))}>
      <select value={spec.vram_type_id || ''} onChange={e => setSpec({ ...spec, vram_type_id: +e.target.value })}>
        <option value="">Выберите тип...</option>
        {refs.vram_types?.map(t => <option key={t.id} value={t.id}><t.name></option>)}
      </select>
    </Field>
    <Field label="Шина (бит) *" type="number" val={spec.memory_bus_bit} set={v => setSpec({ ...spec, memory_bus_bit: +v })}>
    <Field label="TDP (Вт) *" type="number" val={spec.tdp_watts} set={v => setSpec({ ...spec, tdp_watts: +v })}>
    <Field label="Длина (мм) *" type="number" val={spec.length_mm} set={v => setSpec({ ...spec, length_mm: +v })}>
    <Field label="Ширина (мм) *" type="number" val={spec.width_mm} set={v => setSpec({ ...spec, width_mm: +v })}>
    <Field label="PCIe Gen *" val={spec.pcie_gen} set={v => setSpec({ ...spec, pcie_gen: v })} error={specErrors.pcie_gen}>
    <Field label="Питание" val={spec.power_requires} set={v => setSpec({ ...spec, power_requires: v })} placeholder="Выберите тип питания...">
    <Field label="Слоты PCIe" type="number" val={spec.pcie_slots_required} set={v => setSpec({ ...spec, pcie_slots_required: +v })}>
  </div>

  </div>

{/* RAM */}
{slug === 'ram' && <div className="grid grid-cols-2 md:grid-cols-3 gap-4">
  <Field label="Объём (GB) *" type="number" val={spec.total_capacity_gb} set={v => setSpec({ ...spec, total_capacity_gb: +v })}>
  <Field label="Частота (МГц) *" type="number" val={spec.speed_mhz} set={v => setSpec({ ...spec, speed_mhz: +v })}>
  <Field label="Тип ОЗУ *" error={specErrors.ram_type_id} onChange={() => setSpecErrors(p => ({...p, ram_type_id: +e.target.value}))}>
    <select value={spec.ram_type_id || ''} onChange={e => setSpec({ ...spec, ram_type_id: +e.target.value })}>
      <option value="">Выберите тип...</option>
      {refs.ram_types?.map(t => <option key={t.id} value={t.id}><t.name></option>)}
    </select>
  </Field>
  <Field label="Тайминг CL *" type="number" val={spec.latency_cl} set={v => setSpec({ ...spec, latency_cl: +v })}>
  <Field label="Кол-во модулей" type="number" val={spec.modules_count} set={v => setSpec({ ...spec, modules_count: +v })}>
  </div>
}

```

Рисунок 23 – Фрагмент формы динамического ввода характеристик компонента

4 Обработка состояний загрузки и ошибок. Для улучшения пользовательского опыта во всех компонентах, взаимодействующих с API, реализованы состояния загрузки (loading). Пока данные загружаются, пользователю отображаются спиннеры (Loader2 из библиотеки lucide-react). Это особенно важно для страницы профиля и детального просмотра товара, где задержка сети может быть заметна. Пример реализации состояния загрузки на странице детального просмотра товара показан на рисунке 24.

```

if (loading || !data) return <div className="min-h-screen bg-[#0f0f10] flex items-center justify-center text-gray-400">Загрузка...</div>;

```

Рисунок 24 – Индикация загрузки данных на странице детального просмотра товара.

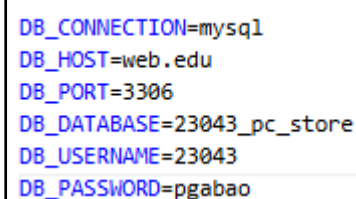
В ходе разработки интерфейса интернет-магазина «ПК» был реализован современный, адаптивный и функциональный пользовательский интерфейс на базе

фреймворка React с использованием утилитарного CSS-фреймворка Tailwind CSS. Применение компонентного подхода, строгая типизация данных через API и глубокая интеграция с серверной логикой Laravel обеспечили высокую производительность приложения и удобство работы как для покупателей, так и для администраторов магазина.

4.3 Разработка веб-приложения

Серверная часть интернет-магазина «ПК» реализована на языке PHP 8.4 с использованием фреймворка Laravel 13. Архитектура приложения построена по паттерну MVC (Model-View-Controller), где Laravel выступает в роли мощного API-сервера. Он обрабатывает входящие HTTP-запросы от клиентской части (React SPA), выполняет бизнес-логику, взаимодействует с базой данных и возвращает ответы в формате JSON.

Конфигурация подключения к СУБД MySQL осуществляется через файл **.env**, параметры из которого автоматически считываются фреймворком при старте приложения. Такой подход позволяет изолировать чувствительные данные (хост, имя базы, логин и пароль) от исходного кода репозитория, что соответствует стандартам безопасности. Основные параметры подключения представлены на рисунке 25.



```
DB_CONNECTION=mysql
DB_HOST=web.edu
DB_PORT=3306
DB_DATABASE=23043_pc_store
DB_USERNAME=23043
DB_PASSWORD=pgabao
```

Рисунок 25 – Фрагмент файла .env с настройками подключения к БД

Маршрутизация API определена в файле `routes/api.php`. Для защиты эндпоинтов и разграничения прав доступа используются встроенные и кастомные `middleware`:

- `auth:sanctum` – проверяет наличие валидного сессионного токена или Bearer-токена, обеспечивая аутентификацию пользователя.

– role:manager, role:admin – кастомные middleware, которые проверяют поле role в модели пользователя и запрещают доступ к административным методам для обычных покупателей и гостей.

Взаимодействие с данными осуществляется через контроллеры, которые инкапсулируют логику обработки запросов. Ключевой особенностью серверной части является работа со сложной структурой товаров. В отличие от простых каталогов, здесь каждый компонент имеет базовые свойства в таблице components и специфические характеристики в отдельных таблицах спецификаций (cpu_specs, gpu_specs).

Контроллер Admin\ComponentController отвечает за управление каталогом. Метод store данного контроллера демонстрирует сложную логику сохранения: при создании товара система сначала создает запись в основной таблице, получает её ID, а затем динамически определяет, в какую таблицу спецификаций необходимо сохранить дополнительные данные, основываясь на выбранной категории компонента. Фрагмент кода метода сохранения представлен на рисунке 26.

```
public function index(Request $request)
{
    $query = Component::with([
        'category',
        'brand',
        'cpuSpec.socket',
        'gpuSpec',
        'ramSpec.ramType',
        'motherboardSpec.socket',
        'motherboardSpec.formFactor',
        'motherboardSpec.ramType',
        'psuSpec',
        'storageSpec',
        'coolerSpec',
        'caseSpec',
        'compatibleSockets:id,name',
        'supportedFormFactors:id,name'
    ]);

    if ($request->has('category')) {
        $query->whereHas('category', fn($q) => $q->where('slug', $request->category));
    }

    if ($request->boolean('in_stock')) {
        $query->where('stock', '>', 0);
    }

    return $query->get();
}

public function show($id)
{
    $component = Component::with([
        'brand', 'category',
        'cpuSpec.socket', 'gpuSpec', 'ramSpec.ramType', 'motherboardSpec.socket', 'motherboardSpec.formFactor',
        'compatibleSockets:id,name',
        'supportedFormFactors:id,name'
    ]->findOrFail($id);
    return response()->json($component);
}
```

Рисунок 26 – Фрагмент ComponentController.php: компонент со спецификациями

Для работы с готовыми сборками (Prebuilt PCs) реализован отдельный контроллер `PrebuiltPcController`. Метод `exclusive` выбирает сборки с определенным тегом, используя возможности Eloquent ORM для фильтрации и жадной загрузки (eager loading) связанных компонентов, чтобы минимизировать количество SQL-запросов (проблема N+1). Это обеспечивает высокую скорость ответа API даже при большом количестве товаров.

На стороне клиента взаимодействие с сервером реализовано через библиотеку `Axios`, настроенную в едином экземпляре `api.js`. Компоненты React отправляют асинхронные запросы к эндпоинтам API. Например, страница профиля (`ProfilePage.jsx`) обновляет данные пользователя через PUT-запрос к `/api/profile`, а configurator получает актуальные цены и наличие через GET-запросы. Ответ сервера всегда стандартизирован: он содержит либо данные в поле `data`, либо сообщение об ошибке в поле `message`, что упрощает обработку исключений на фронтенде.

Пример обработки ответа сервера и отображения данных в компоненте детального просмотра товара представлен на рисунке 27.

```
// Форматирование характеристик в пары label:value
const formatSpecs = () => Object.entries(specs || {}).
  .filter(([_, v]) => v !== null && v !== undefined)
  .filter(([k, _]) => ![ 'created_at', 'updated_at', 'createdAt', 'updatedAt' ].includes(k))
  .map(([k, v]) => {
    // Превращение объекта в строку name/id
    let displayValue = v;
    if (typeof v === 'object' && v !== null) {
      displayValue = v.name || v.id || JSON.stringify(v);
    }

    const labels = {
      socket: "Сокет", cores: "Ядра", threads: "Потоки", base_clock_mhz: "Баз. частота (МГц)", boost_clock_mhz: "Буст. частота (МГц)", tdp_watts: "TDP (Вт)",
      vram_gb: "VRAM (GB)", vram_type: "Тип памяти", memory_bus_bit: "Шина (бит)", length_mm: "Длина (мм)", width_mm: "Ширина (мм)", pcie_slots_required: "Слоты PCIe",
      total_capacity_gb: "Объём (GB)", speed_mhz: "Частота (МГц)", type: "Тип", latency_cl: "Тайминги (CL)", modules_count: "Модулей",
      chipset: "Чипсет", ram_slots: "Слоты [GB]", ram_type: "Тип [GB]", m2_slots: "Слоты M.2", pcie_gen: "Версия PCIe", form_factor: "Форм-фактор",
      wattage: "Мощность (Вт)", efficiency: "Сертификат", modularity: "Модульность", pcie_cables_count: "Кабелей PCIe",
      capacity_gb: "Объём (GB)", read_speed_mbps: "Чтение (МБ/с)", write_speed_mbps: "Запись (МБ/с)",
      tdp_rating_watts: "Рассеивание (Вт)", fan_count: "Вентиляторов",
      case_type: "Тип корпуса", top_fan_slots: "Слоты сверху", fans_included: "[ ] комплекте", drive_bays_3_5: "Отсеки 3.5\"", drive_bays_2_5: "Отсеки 2.5\"",
      front_usb_a: "USB-A", front_usb_c: "USB-C", front_audio_jack: "Аудио", material: "Материал"
    };
    return { label: labels[k] || k, value: typeof displayValue === 'boolean' ? (displayValue ? "Да" : "Нет") : displayValue };
  });

const formattedSpecs = formatSpecs();

return (
  <div className="min-h-screen pt-20 bg-gray-50 dark:bg-[#101019] text-gray-900 dark:text-gray-200 transition-colors duration-300">
    <div className="container mx-auto px-4 py-8 max-w-6xl">
      {/* Навигация назад */}
      <button onClick={() => navigate(-1)} className="flex items-center gap-2 text-gray-600 dark:text-gray-400 hover:text-purple-600 dark:hover:text-purple-400 mb-6 transition-colors">
        <ArrowLeft className="w-5 h-5" /> Назад к каталогу
      </button>

      <motion.div initial={{ opacity: 0, y: 20 }} animate={{ opacity: 1, y: 0 }}>
        {/* Верхний блок: Картинка слева, информация справа (на desktop) */}
        <div className="bg-white dark:bg-[#0f0f10] border border-gray-200 dark:border-white/10 rounded-2xl p-6 mb-6">
```

Рисунок 27 – Фрагмент `ComponentDetailPage.jsx`: форматирование и отображение данных товара

Особое внимание уделено обработке изображений. Файлы загружаются через форму multipart/form-data, валидируются контроллером (проверка типа и размера) и сохраняются в защищенном хранилище storage/app/public. Доступ к ним осуществляется через символическую ссылку, созданную командой `php artisan storage:link`, что позволяет отдавать файлы напрямую веб-сервером без нагрузки на PHP-скрипты.

Таким образом, разработанный бэкенд обеспечивает надежное хранение сложных технических данных, эффективную выборку информации через ORM Eloquent, гибкое управление структурой БД посредством миграций и безопасный обмен данными с клиентским приложением через REST API.

5 Технологическая документация веб-приложения

5.1 Руководство гостя интернет-магазина

Данное руководство предназначено для неавторизованных посетителей (гостей) веб-приложения. Оно описывает базовые сценарии взаимодействия с системой: ознакомление с ассортиментом, поиск комплектующих и готовых сборок, а также формирование предварительного списка покупок.

Для работы с клиентской частью приложения (Single Page Application) гостю необходим любой современный веб-браузер (Google Chrome, Яндекс.Браузер, Microsoft Edge) с включенной поддержкой JavaScript и Cookies. Установка дополнительного программного обеспечения не требуется.

Гость не проходит процедуру входа в систему и работает с ограниченным набором функций. Доступны просмотр каталога, использование configurатора, наполнение корзины, отправка обращения через форму обратной связи, а также регистрация и авторизация. Оформление заказа, просмотр истории покупок и управление личным профилем для гостя недоступны – для этого требуется учётная запись покупателя.

После загрузки приложения пользователь попадает на главную страницу (рисунок 28). В верхней части экрана отображается навигационная панель, доступная на всех страницах. С её помощью выполняется переход между основными разделами: «Каталог», «Конфигуратор», «База знаний», «О нас» и «Корзина». В правой части панели расположены переключатель светлой и тёмной темы оформления, а также элементы входа и регистрации.

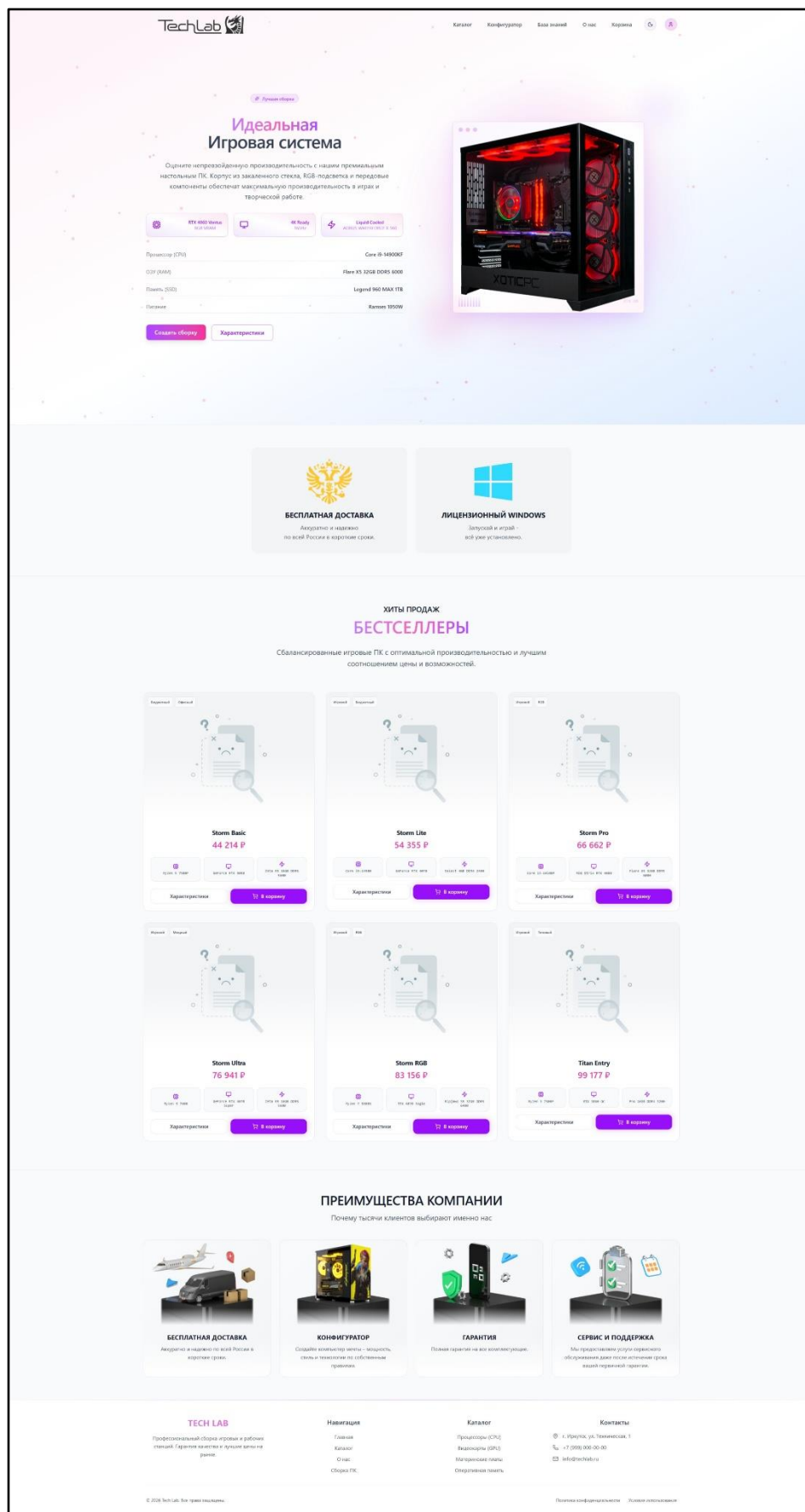


Рисунок 28 – Главная страница

Интерфейс состоит из следующих основных блоков:

- Шапка сайта (Header): содержит логотип «TechLab», основное меню навигации («Комплекующие», «Готовые ПК», «Услуги», «О нас»), строку поиска, а также иконки корзины и профиля (рисунок 29).

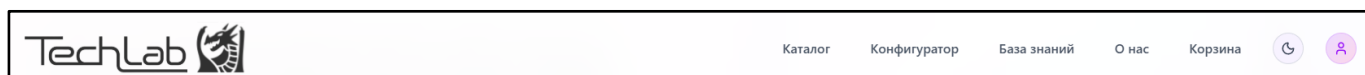


Рисунок 29 – Шапка сайта с навигацией

- Промо-блок (Hero Section): центральный баннер с актуальными новинками и призывом к действию.
- Секция «Эксклюзивные сборки»: витрина флагманских готовых компьютеров с краткими характеристиками и изображениями.
- Подвал (Footer): дублирует навигационные ссылки, содержит информацию о доставке, гарантиях и контакты службы поддержки.

Гость имеет полный доступ к каталогу товаров без необходимости регистрации. Для поиска нужного изделия доступны следующие инструменты:

- Навигация по категориям: выбор раздела (например, «Видеокарты» или «Процессоры») через главное меню или блоки на главной странице.
- Фильтрация: в боковой панели каталога можно отсортировать товары по производителю (бренду), ценовому диапазону и ключевым техническим характеристикам (например, тип сокета или объем памяти).
- Строка поиска: позволяет быстро найти конкретную модель по её названию или артикулу.

При клике на кнопку «Характеристики» в карточке товара открывается модальное окно (рисунок 30). Для отдельных компонентов отображаются кнопки для перехода на страницу с полными техническими спецификациями, статусом наличия на складе и информацией о совместимости (рисунок 31).

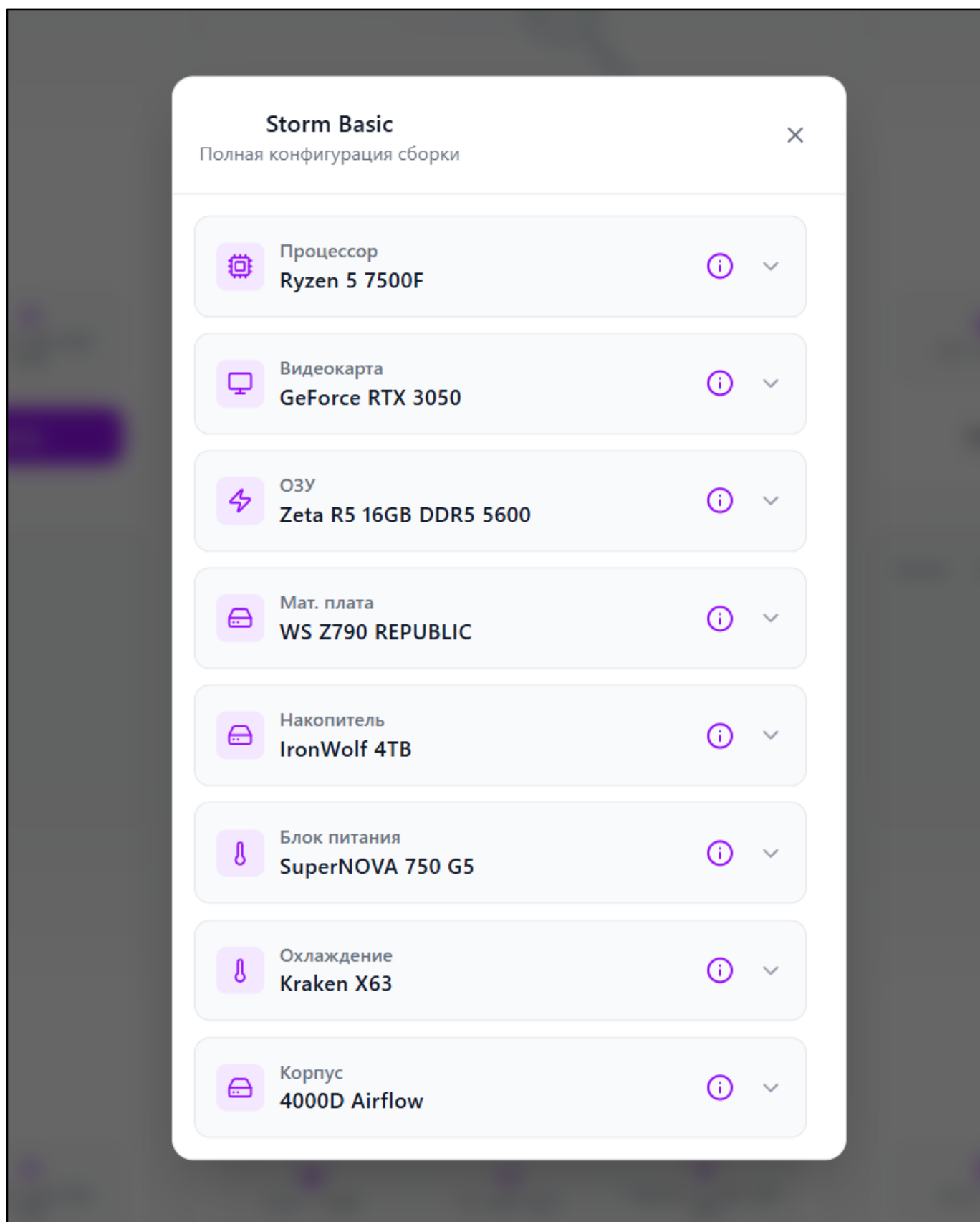


Рисунок 30 – Модальное окно

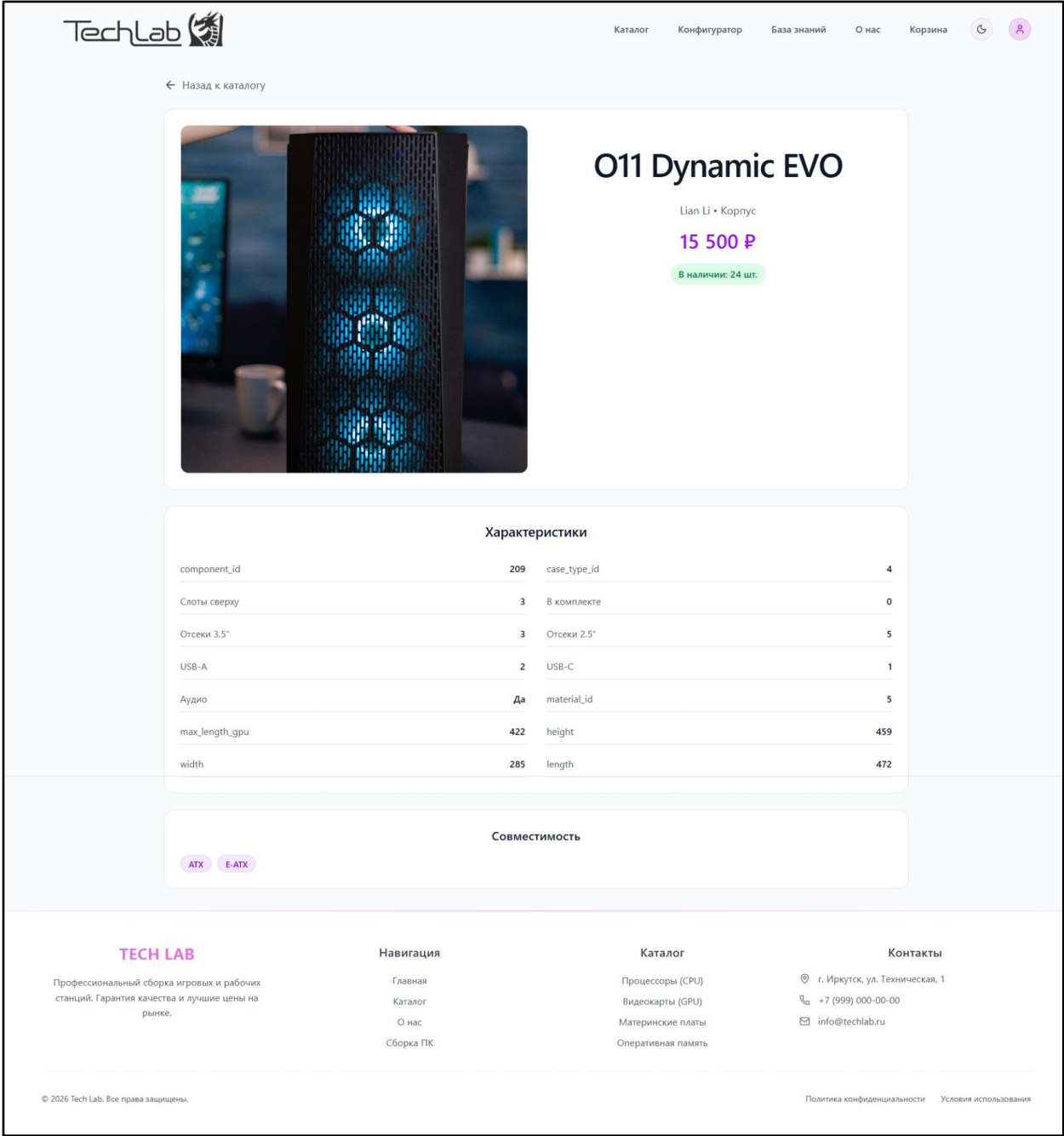


Рисунок 31 – Страница о компоненте

На данном этапе гость может изучить все характеристики, но кнопка добавления в корзину для готовых ПК является основным доступным действием.

Гость может добавлять товары в корзину, нажимая соответствующую кнопку на странице каталога или в карточке товара. Счетчик товаров в иконке корзины (в шапке сайта) обновляется мгновенно без перезагрузки страницы.

Перейдя в корзину, неавторизованный пользователь может:

- Удалить ненужную позицию из списка.

- Просмотреть итоговую сумму заказа.

Содержимое корзины гостя сохраняется на сервере в таблице carts с привязкой к уникальному идентификатору сессии (session_id). При этом поле user_id остаётся равным NULL, что позволяет отличать гостевые корзины от корзин авторизованных пользователей. Такой подход обеспечивает сохранность данных корзины при обновлении страницы и навигации по сайту, а также гарантирует корректный перенос товаров в корзину пользователя после успешной авторизации.

Важным бизнес-правилом системы является то, что гость не может завершить покупку. При попытке нажать кнопку «Оформить заказ» на странице корзины или при клике на иконку «Профиль», система прерывает дальнейшие действия и отображает модальное окно с предложением войти в существующий аккаунт или зарегистрироваться (рисунок 32).

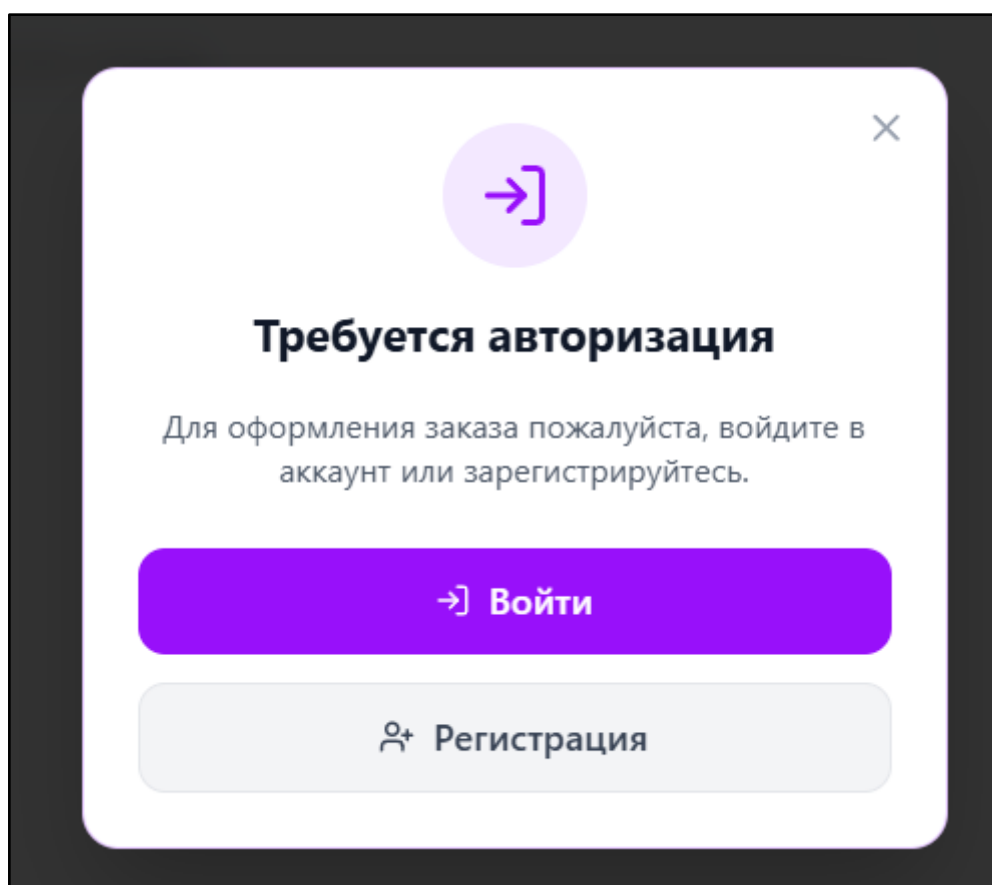


Рисунок 32 – Модальное окно «Требование авторизации»

Дальнейшие сценарии работы (регистрация, оформление заказа, отслеживание статуса доставки и управление личным кабинетом) описаны в разделе 5.2 «Руководство пользователя интернет-магазина».

5.2 Руководство пользователя интернет-магазина

Руководство предназначено для авторизованных покупателей интернет-магазина «ПК». В данном разделе описываются сценарии, недоступные гостю: регистрация и авторизация, перенос гостевой корзины, использование интерактивного конфигуратора совместимых компонентов, оформление заказа с выбором доставки и оплаты, а также работа с личным кабинетом. Базовые сценарии просмотра каталога, фильтрации товаров и добавления их в корзину описаны в разделе 5.1 «Руководство гостя интернет-магазина».

Для оформления заказа, сохранения конфигураций и отслеживания статусов покупок необходимо создать учётную запись. Процесс регистрации инициируется нажатием кнопки «Профиль» или «Войти» в шапке сайта. Система предоставляет форму с полями для ввода имени, адреса электронной почты и пароля (рисунок 33).

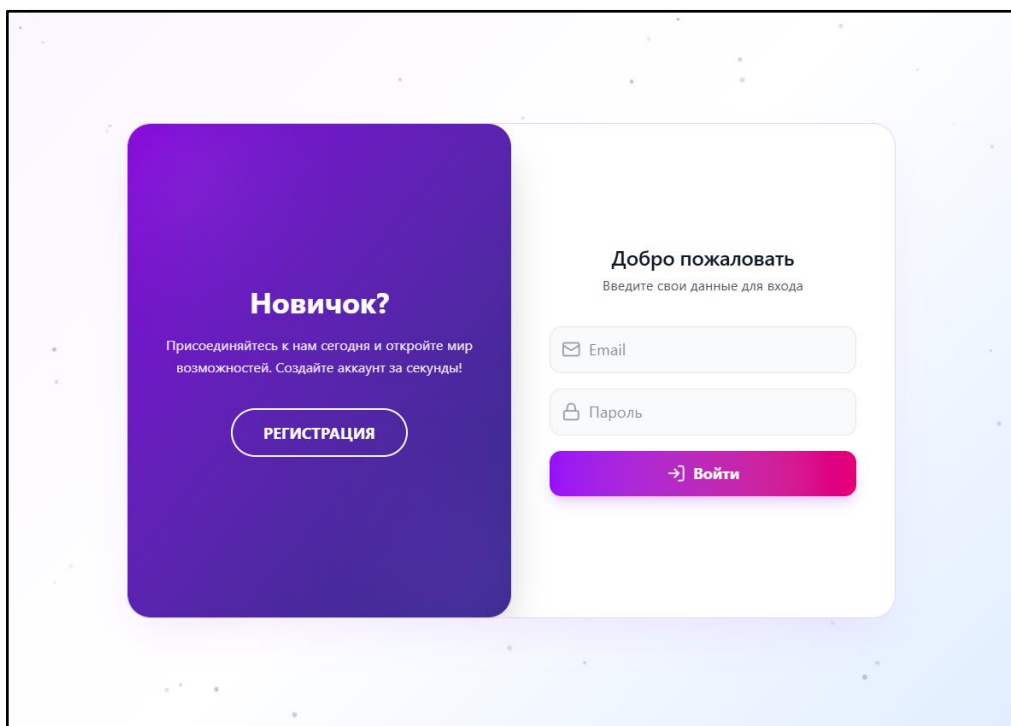


Рисунок 33 – Страница авторизации и регистрации

Для обеспечения безопасности внедрена валидация данных в реальном времени: поле пароля подсвечивается красным при несоответствии требованиям (минимум 8 символов). После успешного создания аккаунта вход в систему осуществляется через форму авторизации путём ввода зарегистрированного email и пароля. Выход из аккаунта выполняется через выпадающее меню пользователя в правом верхнем углу экрана.

Важной особенностью системы является автоматический перенос содержимого гостевой корзины в корзину авторизованного пользователя. Если гость добавил товары в корзину до входа в аккаунт, то сразу после успешной авторизации серверная часть (контроллер CartController) выполняет слияние корзин: записи из таблицы carts, привязанные к session_id гостя, переносятся на user_id вошедшего пользователя.

Пользователю не требуется повторно добавлять товары – содержимое корзины остаётся прежним, но теперь оно сохраняется между сессиями и синхронизируется с сервером. Счетчик товаров в иконке корзины обновляется мгновенно без перезагрузки страницы.

Конфигуратор – ключевой инструмент магазина, позволяющий собрать совместимый компьютер из отдельных компонентов. Доступ к конфигуратору осуществляется через промо блок на главной странице или через прямую ссылку в навигационной панели (рисунок 34).

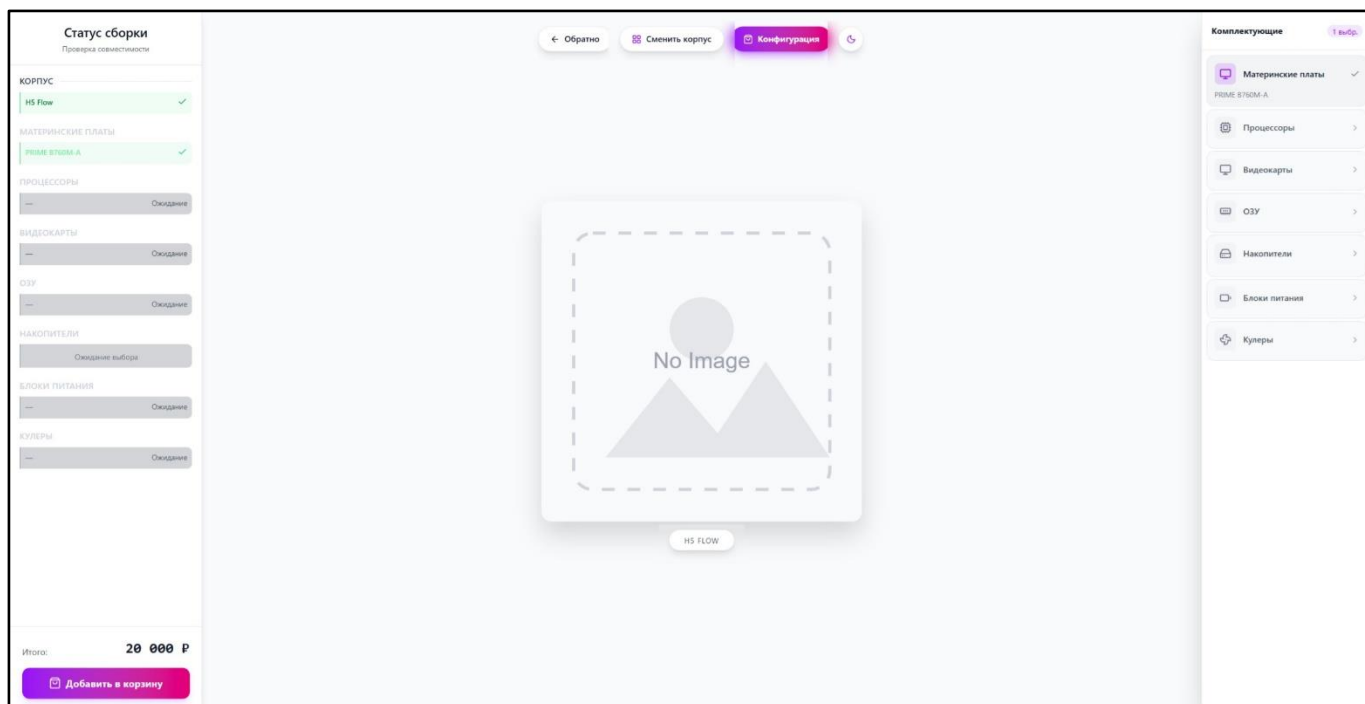


Рисунок 34 – Страница конфигуратора

Интерфейс конфигуратора разделён на две основные области:

- Правая панель – список категорий компонентов (Процессор, Материнская плата, Оперативная память, Видеокарта, Накопитель, Блок питания, Корпус, Охлаждение). Каждая категория отображает текущий статус: выбранный компонент, совместимые альтернативы или недоступные позиции.
- Левая панель – сводка сборки: список выбранных компонентов, индикатор совместимости и итоговая стоимость.

Алгоритм проверки совместимости работает в реальном времени на стороне сервера. При выборе каждого компонента React-клиент отправляет GET-запрос на эндпоинт `/api/configurator/check`, передавая текущий состав сборки. Серверный контроллер `ConfiguratorController` выполняет последовательную проверку по правилам:

- Сокет процессора должен совпадать с сокетом материнской платы.
- Тип оперативной памяти (DDR4/DDR5) должен поддерживаться материнской платой.
- Форм-фактор материнской платы должен помещаться в выбранный корпус.

- Длина видеокарты не должна превышать `max_length_gpu` корпуса.
- Суммарное тепловыделение (TDP) процессора и видеокарты не должно превышать мощность блока питания с запасом 20%.
- Количество слотов M.2 и SATA на материнской плате должно соответствовать числу выбранных накопителей.

Несовместимые компоненты автоматически блокируются в интерфейсе: карточки товаров становятся полупрозрачными, а при наведении появляется подсказка с причиной несовместимости (например, «Требуется сокет LGA1700, выбран AM5 – несовместимый сокет»).

После завершения сборки пользователь может добавить готовую конфигурацию в корзину как единый товар.

Переход к оформлению заказа осуществляется со страницы корзины через кнопку «Оформить заказ». Данная кнопка активна только для авторизованных пользователей и при наличии хотя бы одной позиции в корзине.

Процесс оформления состоит из нескольких шагов:

Выбор места доставки СДЭК, а также заполнение данных получателя. Изначально данные берутся из профиля, однако их можно заменить (рисунок 35).

1

Доставка

2

Оплата

3

Подтверждение

Способ доставки

Пункт выдачи СДЭК

Поиск по названию, адресу или городу...

Показать на карте

СДЭК Бизнес-центр

ул. Тверская, 25

Москва

08:00-20:00

СДЭК Почтамт

ул. Почтовая, 3

Москва

09:00-19:00

Данные получателя

ФИО *

Телефон *

Иванов Иван Иванович

+7 (999) 000-00-00

Email

example@mail.ru

Комментарий к заказу

Пожелания к доставке...

Продолжить →

Рисунок 35 – Страница оформления доставки

Далее идет меню оплаты, где пользователь вносит данные банковской карты (рисунок 36).

Рисунок 36 – Страница оплаты

При успешном завершении транзакции пользователь перенаправляется на страницу подтверждения заказа, где отображается номер заказа, состав покупки и выбранная доставка.

Авторизованный пользователь получает доступ к личному кабинету через иконку профиля в шапке сайта. Кабинет реализован как отдельный маршрут `/profile` с защищённым компонентом `<ProtectedRoute>`, проверяющим наличие валидного токена Sanctum.

Раздел «Профиль» позволяет редактировать персональные данные:

- Имя (поле `name`).
- Адрес электронной почты (поле `email`). При изменении `email` система требует повторного подтверждения через ссылку в письме.
- Пароль. При смене пароля требуется ввод текущего пароля для подтверждения личности. Новый пароль хешируется алгоритмом `bcrypt` перед сохранением в базу данных.

Изменения сохраняются через PUT-запрос на эндпоинт `/api/profile`. При успешном обновлении отображается зелёное уведомление, при ошибке валидации – красное с указанием проблемного поля.

Вкладка «История заказов» предоставляет полный список совершённых покупок (рисунок 37). Каждый заказ отображается в виде карточки с ключевой информацией:

- Дата оформления.
- Итоговая сумма.
- Текущий статус с цветовой индикацией:
- «Оплачен» – синий.
- «В сборке» – фиолетовый.
- «В пути» – оранжевый.
- «Доставлен» – зелёный.
- «Отменён» – красный.

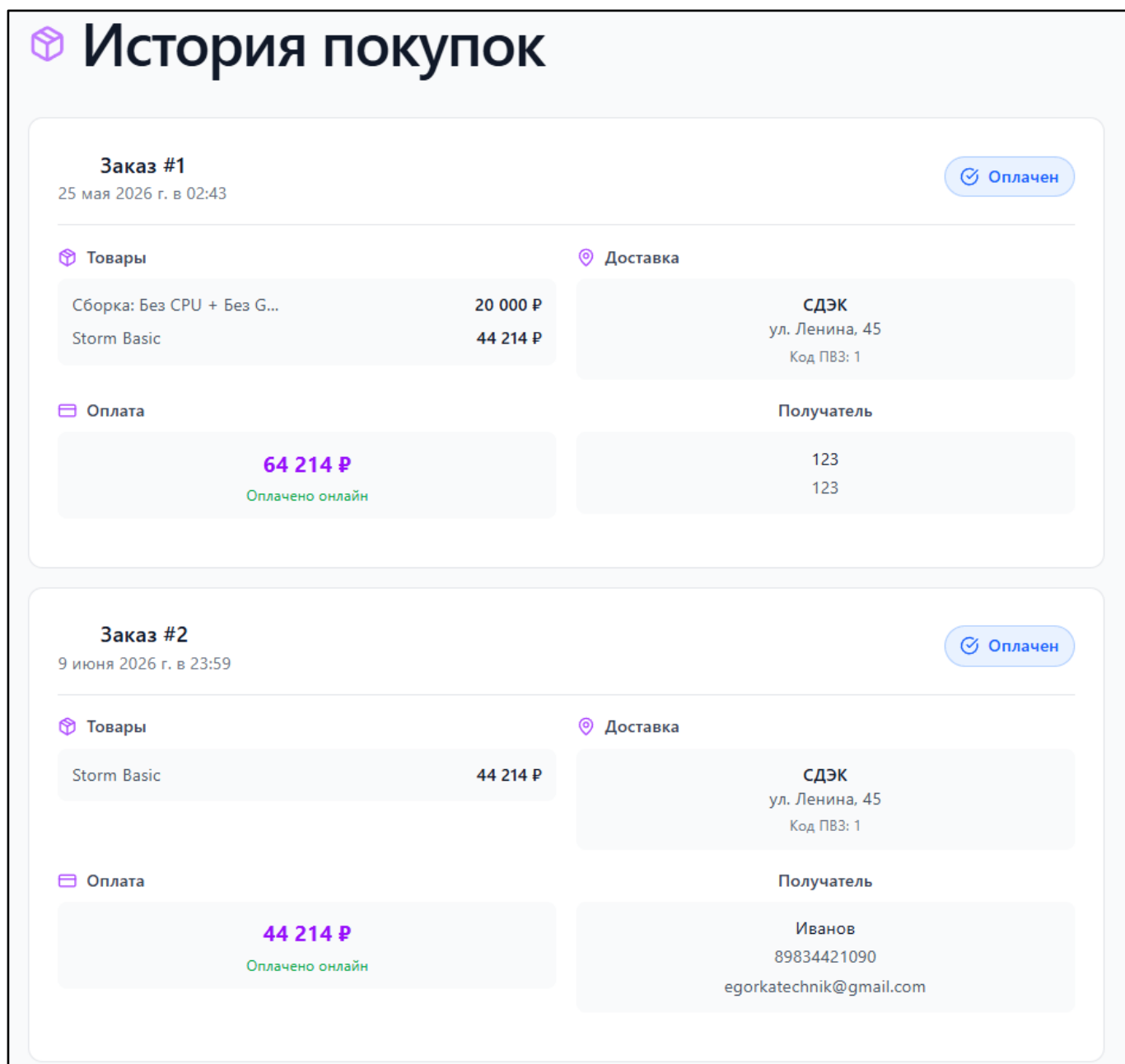


Рисунок 37 – Страница истории заказов

Детальный просмотр конкретного заказа открывается по клику на карточку и содержит:

Полную спецификацию: список купленных компонентов с их характеристиками на момент покупки или модель готового ПК.

Итоговую сумму с разбивкой по позициям.

Выбранный способ доставки и адрес получателя.

Способ оплаты.

Историю изменения статусов с датами.

Помимо каталога и личного кабинета, сайт содержит еще одну информационную страницу, доступную всем пользователям (включая гостей):

Страница «О нас» (рисунок 38) рассказывает историю бренда. Содержит разделы: миссия компании («Создаем компьютеры, которые опережают время»), ключевые преимущества («Гарантия 3 года», «Профессиональная сборка», «Тестирование 24 часа»), блок о команде инженеров и сборщиков с фотографиями, карта с адресом шоу-рума.

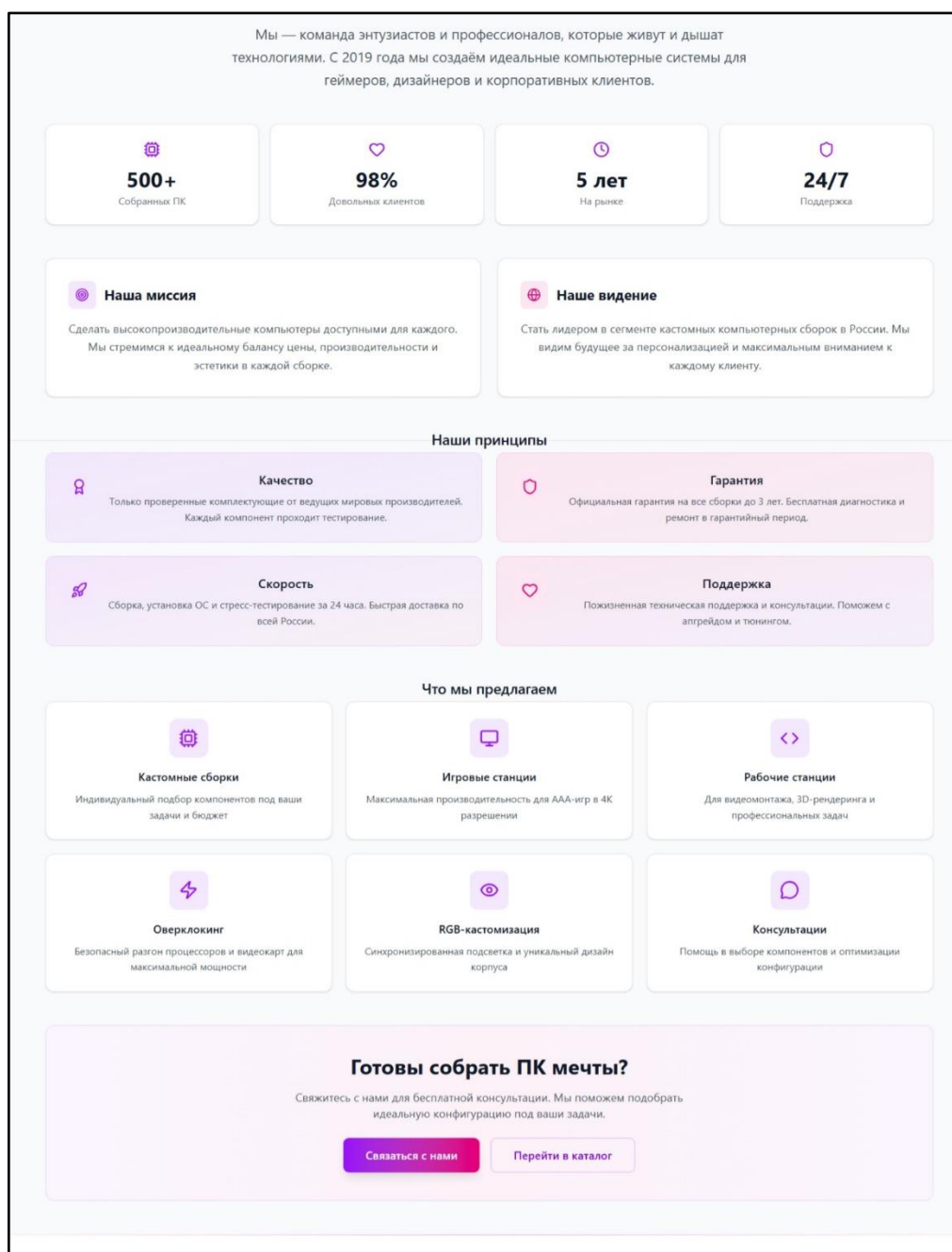


Рисунок 38 – Страница «О нас»

При возникновении технических проблем рекомендуется:

- 1 Проверить стабильность подключения к сети Интернет.
- 2 Очистить кэш браузера (комбинация клавиш Ctrl+Shift+R или Cmd+Shift+R на macOS).
- 3 Убедиться, что в браузере включена поддержка JavaScript и не блокируются сторонние cookies.
- 4 Проверить корректность введенных данных в формах (отсутствие лишних пробелов в email, соответствие пароля требованиям).
- 5 При ошибке 401 (Unauthorized) – выполнить повторный вход в аккаунт, так как срок действия токена Sanctum мог истечь.
- 6 При ошибке 409 (Conflict) при оформлении заказа – проверить наличие товаров на складе, так как они могли быть раскуплены другими пользователями.

При сохранении ошибки следует обратиться в службу поддержки через страницу «Контакты» или направить письмо на электронный адрес магазина, указав текст ошибки, номер заказа (если применимо) и скриншот проблемы.

5.3 Руководство менеджера интернет-магазина

Руководство предназначено для менеджеров интернет-магазина компьютерной техники. Оно описывает порядок доступа к панели управления и основные рабочие сценарии: управление сложным каталогом комплектующих, обработка входящих заказов, работа с готовыми сборками и контроль складских остатков.

Доступ к административному интерфейсу осуществляется по маршруту /admin. Вход выполняется через стандартную форму авторизации сайта. Система автоматически перенаправляет пользователя в панель управления только при наличии в учетной записи роли manager или admin. Для гостей и обычных покупателей доступ к данным разделам заблокирован на уровне клиентской маршрутизации (React Router) и серверных middleware (Laravel Sanctum).

После успешной авторизации менеджер попадает на главную страницу панели – дашборд (рисунок 39). Верхняя часть экрана содержит информационные карточки с ключевыми метриками в реальном времени: общее количество товаров в каталоге, общее количество компонентов, а также компоненты, запас которых подходит к концу. Слева расположена панель для переключения на соответствующие разделы доступные для менеджера: дашборд, компоненты, готовые ПК и заказы.

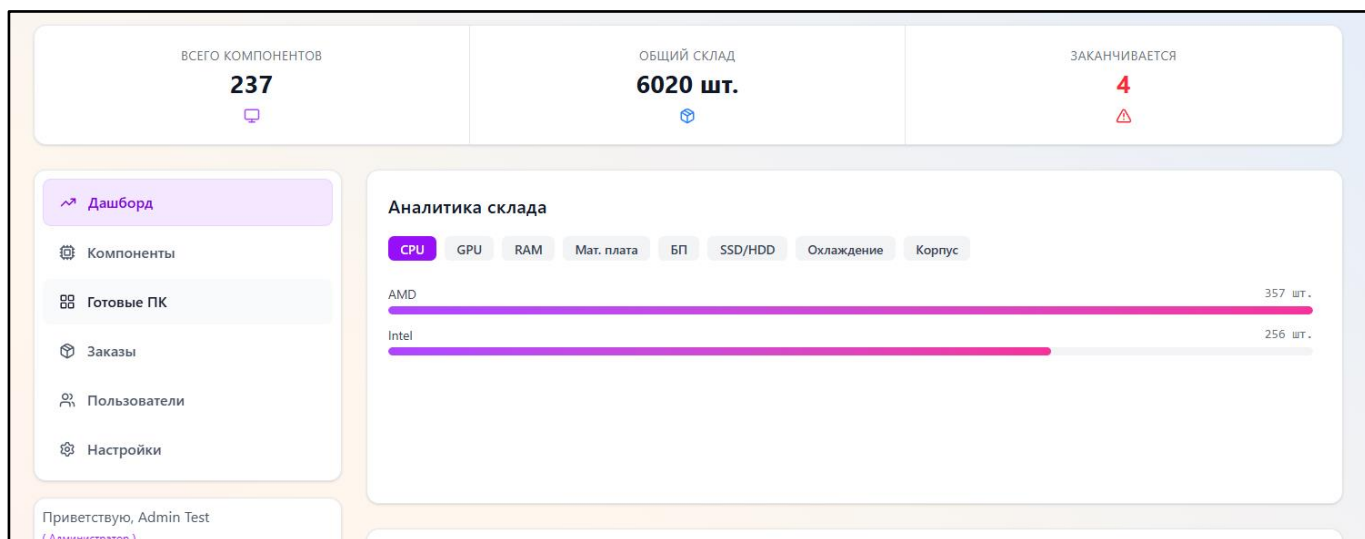


Рисунок 39 – Страница дашборда

Раздел компоненты является основным инструментом формирования ассортимента (рисунок 40). Учитывая сложную структуру данных магазина, интерфейс предоставляет расширенные возможности:

- Фильтрация и поиск: осуществляется по названию модели, категории компонента и по малому остатку на складе.
- Таблица товаров: отображает базовую информацию (номер, название, категория, цена, остаток на складе).
- Динамическое создание и редактирование: при добавлении нового товара менеджер выбирает категорию. В зависимости от выбора форма динамически меняет свой состав:
 - 1 Для процессоров появляются поля: сокет, количество ядер, частоты, TDP.
 - 2 Для видеокарт: объём видеопамяти, тип памяти, длина карты, требования к питанию.

3 Для материнских плат: сокет, форм-фактор, количество слотов RAM и М.2.

– Загрузка изображений: реализована загрузка основного изображения компонента. Файлы сохраняются в защищенном хранилище и становятся доступны по публичной ссылке.

Дашборд	Поиск...	Все типы	Наличие	Excel	Добавить
ID	Модель	Тип	Цена	Наличие	Действия
#237	NV7	Корпуса	21500.00 Р	11 шт.	
#236	Pure Base 500DX	Корпуса	9000.00 Р	25 шт.	
#235	Torrent	Корпуса	17500.00 Р	18 шт.	
#234	LANCOOL III	Корпуса	16500.00 Р	6 шт.	
#233	275R Airflow	Корпуса	7500.00 Р	21 шт.	
#232	H1 V2	Корпуса	14500.00 Р	21 шт.	
#231	C200 Glass	Корпуса	6500.00 Р	6 шт.	
#230	ROG Strix Helios	Корпуса	22500.00 Р	7 шт.	
#229	MPG Velox 100P	Корпуса	8500.00 Р	6 шт.	
#228	CL500	Корпуса	5000.00 Р	23 шт.	
Показывать по: 10 25 50					
Стр. 1 из 24					

Рисунок 40 – Страница компонентов

Управление готовыми сборками (Prebuilt PCs) это отдельный раздел, посвященный управлению готовыми компьютерами (рисунок 41). Здесь менеджер создает витринные предложения, объединяя отдельные компоненты в единую конфигурацию:

– Конфигуратор сборки: при создании ПК менеджер выбирает конкретные компоненты из базы данных. Система автоматически рассчитывает итоговую стоимость как сумму цен выбранных частей.

– Для готовой сборки можно загрузить изображения собранного компьютера, в случае если изображение не будет предоставлено, то система будет использовать изображение используемого в сборке корпуса.

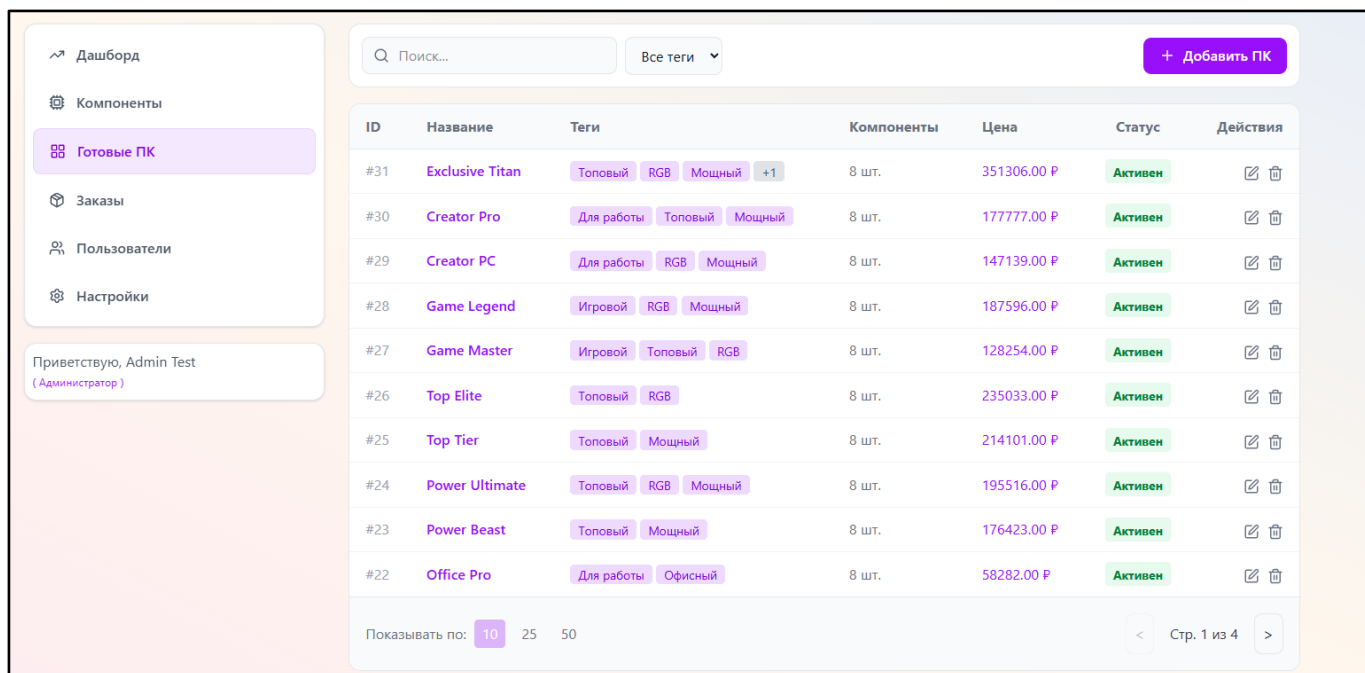


Рисунок 41 – Страница готовых ПК

В разделе «Заказы» менеджер управляет жизненным циклом покупок. Список заказов содержит информацию о номере транзакции, клиенте, сумме и текущем статусе (рисунок 42).

- При клике на заказ открывается детальное представление. Для заказов с готовыми ПК отображаться полная конфигурация системы. Для заказов с отдельными компонентами – список позиций с их характеристиками.

- Менеджер может изменять статус заказа через соответствующие кнопки в заказе («Оплачен», «В сборке», «В пути», «Доставлен», «Отменен»).

Создан: 25.05.2026, 02:43:07

Товары в заказе

Сборка: Без CPU + Без GPU

Конфигуратор

20 000 Р

х1 шт.

Комплектующие:

КОРПУС

H5 Flow

8 500 Р

1 шт.

МАТЕРИНСКАЯ ПЛАТА

PRIME B760M-A

11 500 Р

1 шт.

Storm Basic

Готовая сборка

44 214 Р

х1 шт.

Комплектующие:

КОРПУС

4000D Airflow

9 500 Р

1 шт.

ПРОЦЕССОР

Ryzen 5 7500F

17 990 Р

1 шт.

МАТЕРИНСКАЯ ПЛАТА

WS Z790 REPUBLIC

62 500 Р

1 шт.

ОПЕРАТИВНАЯ ПАМЯТЬ

Zeta R5 16GB DDR5 5600

7 200 Р

1 шт.

ОХЛАЖДЕНИЕ

Kraken X63

12 500 Р

1 шт.

ВИДЕОКАРТА

GeForce RTX 3050

22 990 Р

1 шт.

БЛОК ПИТАНИЯ

SuperNOVA 750 GS

9 000 Р

1 шт.

НАКОПИТЕЛЬ

IronWolf 4TB

8 500 Р

1 шт.

Покупатель

ФИО

123

ТЕЛЕФОН

123

Аккаунт

Admin Test

admin@test.com

Доставка

ТИП

Самовывоз (СДЭК)

АДРЕС

ул. Ленина, 45

КОД ПВЗ

1

Управление

Статус

Оплачен

Сборка

Отправлен

Доставлен

Отменён

Рисунок 42 – Страница управления готовыми сборками

Руководство предназначено для администраторов интернет-магазина компьютерной техники. Администратор обладает максимальным уровнем привилегий в системе, включая в себя функциональные возможности роли «Менеджер». В разделе детализированы исключительно те сценарии и инструменты, которые доступны только администратору и направлены на глобальное управление системой, обеспечение безопасности и контроль учетных данных.

полная таблица зарегистрированных клиентов и сотрудников (рисунок 43).

Администратор может выполнять следующие действия:

- Изменение ролей: повышение привилегий обычного пользователя до роли «Менеджер», например, при приеме нового сотрудника на работу или понижение роли при необходимости.
- Удаление пользователей.

ID	ИМЯ	EMAIL	РОЛЬ	ДЕЙСТВИЯ
1	Admin Test	admin@test.com	Администратор	—
2	Manager Test	manager@test.com	Менеджер	 
3	User 1	user1@test.com	Пользователь	 
4	User 2	user2@test.com	Пользователь	 
5	User 3	user3@test.com	Пользователь	 
6	User 4	user4@test.com	Пользователь	 
7	User 5	user5@test.com	Пользователь	 
8	User 6	user6@test.com	Пользователь	 
9	User 7	user7@test.com	Пользователь	 

Рисунок 43 – Страница управления пользователями

Администратор отвечает за визуальное представление магазина, а именно за управление промо-блоком (Hero Section). Администратор определяет, какие готовые сборки будут помечены тегом «Эксклюзив» для вывода в приоритетный блок на главной странице.

В то время как менеджер наполняет справочники конкретными значениями (добавляет новый бренд или модель), администратор управляет самой структурой классификаторов. Это включает в себя добавление новых типов спецификаций

(например, введение нового стандарта форм-фактора или типа разъема), что требует обновления логики конфигуратора совместимости (рисунок 44). Данные изменения критически важны для поддержания актуальности базы знаний системы при выходе нового аппаратного обеспечения.

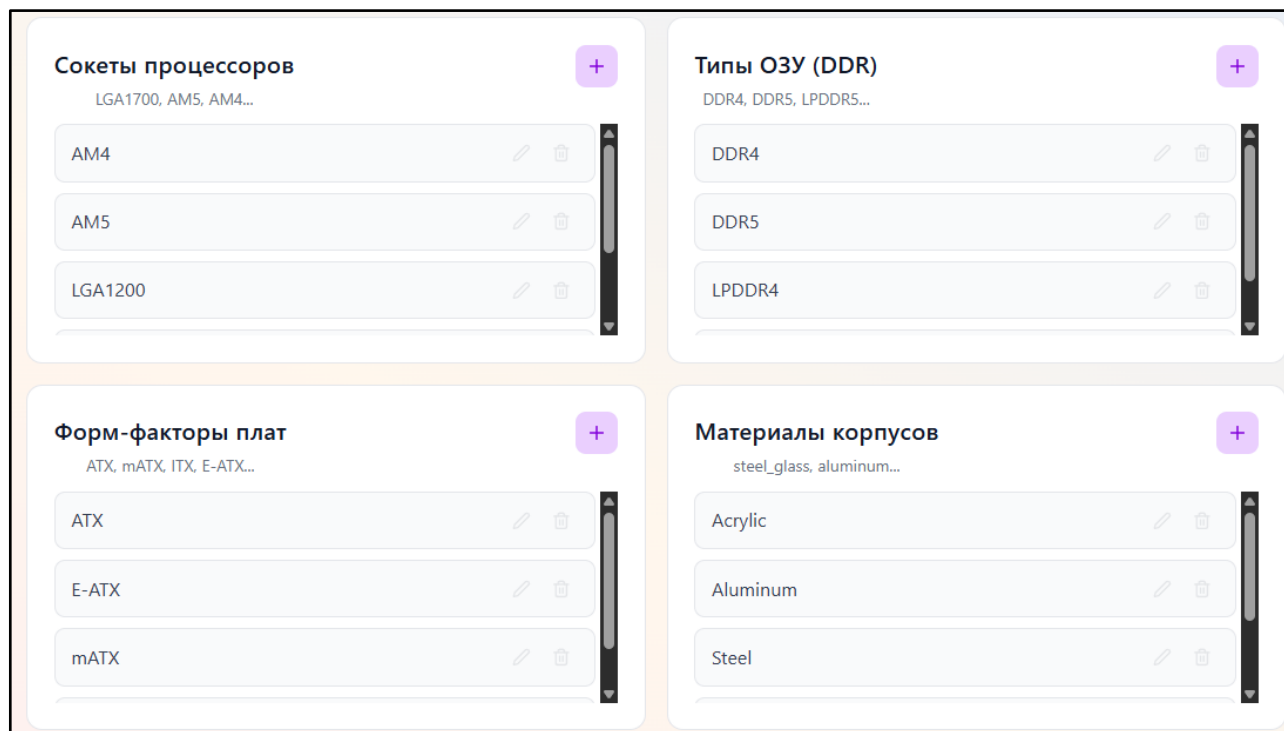


Рисунок 44 – Управление справочниками

Все маршруты административной панели защищены middleware auth:sanctum и проверкой ролей, что исключает несанкционированный доступ. Данные хранятся с соблюдением принципов целостности: изменение цены компонента в каталоге не влияет на цену в уже оформленных заказах (цена фиксируется в момент покупки).

Таким образом, роль администратора сфокусирована на стратегическом управлении платформой, контроле доступа и обеспечении технической целостности системы, в то время как операционная работа с товарами и заказами делегирована роли менеджера.

Заключение

В рамках данного курсового проекта было спроектировано и разработан интернет-магазин компьютерной техники, предназначенный для автоматизации процессов продажи электроники, управления сложным каталогом товаров с техническими характеристиками и обработки клиентских заказов.

На этапе предпроектного исследования была проанализирована предметная область рынка компьютерной техники, определены ключевые бизнес-процессы и сформулированы функциональные требования. Был обоснован выбор современного технологического стека: серверная часть реализована на фреймворке Laravel 13 (PHP 8.4) с использованием паттерна MVC и ORM Eloquent, а клиентская часть построена как одностраничное приложение (SPA) на базе библиотеки React 18 с применением функциональных компонентов, хуков и утилитарного CSS-фреймворка Tailwind CSS.

В ходе проектирования были разработаны UML-диаграммы (вариантов использования, потоков данных, классов и последовательности), детально описывающие логику работы системы. Спроектирована реляционная база данных в СУБД MySQL, приведённая к третьей нормальной форме (3НФ). Особое внимание уделено архитектуре хранения данных: реализована нормализованная структура с вынесением специфических технических характеристик процессоров, видеокарт и других компонентов в отдельные таблицы спецификаций, связанные с основной таблицей товаров. Также предусмотрена поддержка готовых компьютерных сборок и справочников совместимости (сокет, форм-факторы, типы памяти).

Практическая реализация проекта включала следующие этапы:

Разработку защищенного REST API для асинхронного взаимодействия фронтенда и бэкенда с использованием библиотеки Axios.

- Создание адаптивного пользовательского интерфейса с темной темой оформления, обеспечивающего корректное отображение на мобильных устройствах и десктопных ПК.

- Реализацию системы аутентификации и авторизации с разграничением прав доступа на роли «Гость», «Покупатель», «Менеджер» и «Администратор» с использованием токенов Laravel Sanctum.
- Внедрение функционала корзины, оформления заказа с фиксацией цены и состава на момент покупки, а также личного кабинета с историей транзакций.
- Разработку расширенной административной панели для управления каталогом компонентов (с динамическими формами спецификаций), готовыми сборками, заказами и пользователями, включая экспорт отчетов в формат Excel.
- Организацию хранения изображений товаров в защищенном хранилище с доступом через символические ссылки.

Разработанный интернет-магазин полностью соответствует требованиям технического задания, обеспечивает надежное хранение данных, защиту от распространенных веб-уязвимостей (SQL-инъекций, XSS, CSRF) и высокий уровень пользовательского опыта (UX). Полученные в ходе выполнения проекта навыки анализа предметной области, проектирования архитектуры сложных данных, разработки full-stack приложений на связке Laravel + React и написания технической документации являются важной компетенцией для дальнейшей профессиональной деятельности в сфере современной веб-разработки.

Список используемых источников

- 1 ГОСТ 34.602-2020. Информационная технология. Комплекс стандартов на автоматизированные системы. Техническое задание на создание автоматизированной системы: национальный стандарт Российской Федерации: дата введения 2021-03-01. – Москва: Стандартинформ, 2020. – 14 с. – Текст: непосредственный.
- 2 ГОСТ Р 56477-2015. Проектирование и внедрение информационных систем. Общие требования: национальный стандарт Российской Федерации: дата введения 2016-01-01. – Москва: Стандартинформ, 2015. – 12 с. – Текст: непосредственный.
- 3 Banks, A. React and Redux: Functional Web Development / A. Banks, E. Porcello. – Boston: O'Reilly Media, 2020. – 336 p. – ISBN 978-1-492-05136-4. – Текст: непосредственный.
- 4 Flanagan, D. JavaScript: The Definitive Guide / D. Flanagan. – 7th ed. – Sebastopol: O'Reilly Media, 2020. – 1096 p. – ISBN 978-1-491-95202-3. – Текст: непосредственный.
- 5 Stauffer, M. Laravel Up & Running: A Framework for Building Modern PHP Apps / M. Stauffer. – 3rd ed. – Sebastopol: O'Reilly Media, 2023. – 380 p. – ISBN 978-1-098-13626-6. – Текст: непосредственный.
- 6 Axios HTTP Client: official documentation [Electronic resource]. – URL: <https://axios-http.com/docs/intro> (дата обращения: 20.05.2026). – Текст: электронный.
- 7 Laravel 11.x Documentation: official website [Electronic resource]. – URL: <https://laravel.com/docs/11.x> (дата обращения: 20.05.2026). – Текст: электронный.
- 8 MySQL 8.4 Reference Manual: official website [Electronic resource]. – URL: <https://dev.mysql.com/doc/refman/8.4/en/> (дата обращения: 20.05.2026). – Текст: электронный.
- 9 React 18 Documentation: official website [Electronic resource]. – URL: <https://react.dev/learn> (дата обращения: 20.05.2026). – Текст: электронный.

- 10 Tailwind CSS v3 Documentation: official website [Electronic resource]. – URL: <https://tailwindcss.com/docs> (дата обращения: 20.05.2026). – Текст: электронный.
- 11 Vite Build Tool: official documentation [Electronic resource]. – URL: <https://vitejs.dev/guide/> (дата обращения: 20.05.2026). – Текст: электронный.
- 12 Купер, А. Об интерфейсе. Основы проектирования взаимодействия / А. Купер. – 4-е изд. – Санкт-Петербург: Символ-Плюс, 2019. – 720 с. – ISBN 978-5-93286-205-6. – Текст: непосредственный.
- 13 Маркотт, И. Отзывчивый веб-дизайн / И. Маркотт; пер. с англ. – Москва: Манн, Иванов и Фербер, 2018. – 240 с. – ISBN 978-5-00100-641-9. – Текст: непосредственный.
- 14 Норман, Д. Дизайн привычных вещей / Д. Норман; пер. с англ. – 2-е изд. – Москва: Вильямс, 2020. – 384 с. – ISBN 978-5-8459-2126-8. – Текст: непосредственный.
- 15 Смирнов, А. В. HTML5 и CSS3. Разработка сайтов для любых браузеров и устройств / А. В. Смирнов. – Санкт-Петербург: Питер, 2021. – 352 с. – ISBN 978-5-4461-1311-5. – Текст: непосредственный.
- 16 Файзрахманов, А. Архитектура сложных веб-приложений. С примерами на Laravel / А. Файзрахманов. – 2-е изд. – [Б. м.]: Самиздат, 2020. – URL: https://github.com/adelf/acwa_book_ru (дата обращения: 20.05.2026). – Текст: электронный.
- 17 Флэнаган, Д. JavaScript: Подробное руководство / Д. Флэнаган; пер. с англ. – 7-е изд. – Санкт-Петербург: Символ-Плюс, 2020. – 1080 с. – ISBN 978-5-93286-103-5. – Текст: непосредственный.
- 18 Щеголев, А. В. Современные технологии веб-разработки: учебное пособие / А. В. Щеголев. – Москва: ИНФРА-М, 2022. – 288 с. – ISBN 978-5-16-123456-7. – Текст: непосредственный.

Приложение А Техническое задание

Министерство образования Иркутской области

Государственное бюджетное профессиональное

образовательное учреждение Иркутской области

«Иркутский авиационный техникум»

(ГБПОУИО «ИАТ»)

Техническое задание

ИНТЕРНЕТ-МАГАЗИН «ПК»

Руководитель:

(подпись, дата)

(Н.Р. Огородникова)

Студент:

(подпись, дата)

(Е.А. Попов)

Иркутск, 2026

1 Введение

1.1 Общие сведения

Документ представляет собой техническое задание на создание интернет-магазина компьютерной техники, предназначенного для автоматизации процессов продажи персональных компьютеров, комплектующих и оказания услуг по сборке. Система позволяет управлять сложным каталогом товаров с детальными техническими характеристиками, обрабатывать заказы и администрировать контент.

Вид программного продукта: интернет-магазин.

1.2 Цели и задачи

Целью создания системы является автоматизация бизнес-процессов продажи компьютерной техники, повышение удобства выбора совместимых компонентов для клиентов и обеспечение эффективного инструментария для управления ассортиментом и заказами.

Задачи интернет-магазина включают:

- Разработку системы управления пользователями с четким разграничением прав доступа на четыре основные роли: «Гость» (ограниченный просмотр), «Покупатель» (оформление заказов), «Менеджер» (обработка заказов и наполнение каталога) и «Администратор» (полный контроль над системой).
- Реализацию функционала каталога товаров, поддерживающего многоуровневую структуру данных: базовые характеристики товара и специфические технические параметры (сокет, частоты, форм-факторы), хранящиеся в связанных таблицах спецификаций.
- Внедрение модуля корзины и оформления заказов, позволяющего пользователю формировать список покупок как из отдельных компонентов, так и из готовых сборок, а также сохранять историю транзакций.
- Создание модуля готовых компьютерных сборок (Prebuilt PCs) с возможностью отметки «Эксклюзив».

- Реализацию функционала корзины и оформления заказов с фиксацией состава и стоимости на момент покупки.
- Обеспечение адаптивного интерфейса для всех типов устройств.
- Создание защищенной административной панели с функциями экспорта отчетов в формат Excel и динамическими формами ввода данных, меняющимися в зависимости от категории товара.

2 Основания для разработки

2.1 Нормативные документы

Документ основывается на следующих нормативных документах:

- ГОСТ 34.602-2020 «Информационная технология. Комплекс стандартов на автоматизированные системы. Техническое задание на создание автоматизированной системы».
- ГОСТ Р 56477-2015 «Проектирование и внедрение информационных систем. Общие требования».
- Приказ №-у от 24.01.2026 г. «Об утверждении тем курсовых проектов».
- Методические указания по выполнению курсового проектирования.

2.2 Проектные документы

Проектные документы включают:

- Задание на курсовое проектирование.
- Руководство пользователя.

3 Назначение системы

3.1 Общее описание

Интернет-магазин предназначен для функционирования в сети Интернет и обеспечивает выполнение следующих функций:

- Для покупателей: Поиск и фильтрация комплектующих по техническим характеристикам, просмотр детальной информации о совместимости, конфигурирование ПК, добавление товаров в корзину, оформление заказа, отслеживание статуса сборки и доставки в личном кабинете.

- Для менеджеров и администраторов: Управление каталогом компонентов (включая ввод специфических характеристик), создание готовых сборок, обработка заказов (смена статусов, проверка оплаты), управление пользователями и справочниками совместимости (сокет, типы памяти).

Система поддерживает процессы электронной коммерции, включая учет складских остатков, формирование финансовой истории заказов, экспорт отчетов в Excel и коммуникацию с клиентами.

4 Требования к системе

4.1 Функциональные требования

1 Функционал роли «Гость» (Неавторизованный пользователь):

- Просмотр главной страницы, промо-блоков и раздела «Эксклюзивные сборки».

- Навигация по каталогу компонентов и готовых ПК.

- Применение фильтров по категориям, брендам, цене и техническим характеристикам.

- Поиск товаров по названию и артикулу.

- Просмотр детальной карточки товара с полными спецификациями.

- Проверка наличия товара на складе.

- Добавление товаров в корзину (временное хранение).

- Просмотр содержимого корзины.

- Регистрация новой учетной записи.

- Авторизация в системе.

- Отправка сообщений через форму обратной связи.

2 Функционал роли «Пользователь»:

- Все функции роли «Гость», а также:
- Оформление заказа с выбором способа доставки и оплаты.
- Сохранение адресов доставки в личном профиле.
- Просмотр истории всех совершенных заказов.
- Отслеживание текущего статуса заказа («Новый», «В сборке», «Доставлен»).

- Редактирование личных данных (имя, email).
- Управление списком сохраненных адресов.
- Выход из учетной записи.

3 Функционал роли «Менеджер»:

- Все функции роли «Покупатель», а также:
- Доступ к административной панели.
- CRUD-операции (создание, чтение, обновление, удаление) с компонентами каталога.

- Ввод и редактирование технических спецификаций для различных типов компонентов.

- Управление готовыми сборками ПК (конфигурирование, установка цены, загрузка фото).

- Управление заказами: просмотр списка, изменение статусов, просмотр деталей состава заказа.

- Экспорт каталога и отчетов по заказам в формат Excel.
- Управление справочниками (бренды, категории).

4 Функционал роли «Администратор»:

- Все функции роли «Менеджер», а также:
- Управление пользователями: просмотр списка, изменение ролей (назначение менеджеров), блокировка аккаунтов.

- Доступ к системным настройкам и логам приложения.
- Полное управление контентом главной страницы.

4.2 Технические требования

4.2.1 Надежность и целостность данных

- Валидация пользовательского ввода на клиенте: Все формы, поля, файлы и параметры должны проверяться интерфейсом перед отправкой запроса на сервер (проверка формата email, сложности пароля, заполнения обязательных полей).
- Защита от SQL-инъекций: Использование ORM Eloquent с параметризованными запросами.
- Транзакционность: Оформление заказа должно выполняться в рамках транзакции БД (гарантия создания записи заказа и его позиций одновременно или отката при ошибке).
- Обработка ошибок: Система должна корректно обрабатывать исключения (например, отсутствие товара на складе в момент заказа) и возвращать понятные сообщения пользователю без раскрытия внутренней структуры сервера.

4.3 Эксплуатационные требования

1 Среда функционирования:

- Поддержка современных браузеров: Яндекс.Браузер, Google Chrome, Microsoft Edge.
- Адаптивность: Корректное отображение интерфейса на устройствах с шириной экрана от 375px (мобильные телефоны) до 1920px и выше (Full HD мониторы).
- Темная тема оформления как основная.

2. Установка и развёртывание:

- Наличие инструкции по установке зависимостей (Composer, npm), настройке .env и миграции базы данных.

4.4 Требования к информационной безопасности

Интернет-магазин должен обеспечивать защиту данных и предотвращение несанкционированного доступа в соответствии со следующими требованиями:

1 Аутентификация и авторизация:

- Система обеспечивает строгую аутентификацию пользователей перед предоставлением доступа к защищенным ресурсам личного кабинета и административной панели.
- Вход в систему осуществляется по связке адреса электронной почты и пароля.
- Пароли всех пользователей хранятся в базе данных исключительно в хешированном виде с применением стойкого криптографического алгоритма Bcrypt, что исключает возможность восстановления исходных значений при компрометации базы.
- Для каждой активной сессии пользователя формируется уникальный API-токен на базе пакета Laravel Sanctum. Токен передается в заголовке каждого HTTP-запроса и имеет ограниченное время жизни.

2 Защита от распространенных уязвимостей:

- Защита от межсайтового скриптинга (XSS): обеспечивается автоматическим экранированием всех выводимых данных библиотекой React на клиентской стороне и валидацией входных строк контроллерами Laravel на серверной стороне.
- Защита от межсайтовой подделки запроса (CSRF): реализуется посредством проверки заголовков авторизации и использования сессионных токенов для критических операций.
- Защита от SQL-инъекций: достигается за счет использования ORM Eloquent, которая автоматически очищает пользовательский ввод от специальных символов.
- Любые данные, поступающие от клиента, проходят строгую серверную валидацию, где проверяются типы данных, диапазоны значений и форматы. Ошибки валидации возвращаются пользователю в стандартизированном формате JSON.

5 Требования к техническому обеспечению

5.1 Сервер (Минимальные требования для развертывания)

- Процессор: 2 ядра, частота от 2.4 ГГц.
- Оперативная память: 4 Гб.
- Дисковое пространство: 50 Гб (SSD рекомендовано).
- Сеть: доступ к Интернет, порт 8000 (или 80/443).

5.2 Хранилище данных

- Тип накопителей: SSD;
- Общий объем хранилища: 50 Гб;
- IOPS на чтение: 10000;
- IOPS на запись: 5000.

5.3 Клиентские устройства

5.3.1 Минимальные требования для персонального компьютера (ноутбука)

- Количество ядер процессора: 2 ядра.
- Тактовая частота процессора: 2.0 ГГц.
- Оперативная память: 4 Гб.
- Тип диска: HDD или SSD.
- Разрешение дисплея: Full HD (1920×1080).
- Пропускная способность сети: 10 Мбит/с.

5.3.2 Минимальные требования для смартфона

- Количество ядер процессора: 4 ядра;
- Тактовая частота процессора: 1.8 ГГц;
- Оперативная память: 3 Гб;
- Свободного места на хранилище: не менее 100 МБ;
- Диагональ экрана: 4.7 дюймов;

- Разрешение экрана: 1280×720 пикселей;
- Ориентация экрана: поддержка портретной и альбомной;
- Навигация: сенсорная.

5.3.3 Сетевые требования

- Доступ к сети Интернет со скоростью не менее 10 Мбит/с.
- Поддержка следующих сетевых протоколов: HTTP/1.1, HTTP/2, DNS, TCP/IP.
- Поддержка IPv4.
- Возможность работы через порты 80 (HTTP) и 443 (HTTPS).

6 Требования к программному обеспечению

6.1 Сервер

- Операционная система: Windows 10/11 или Linux (Ubuntu 20.04+).
- Сервер базы данных: MySQL 8.4.
- Веб-сервер: Apache 2.4 или Nginx.
- Интерпретатор языка программирования: PHP версии 8.4 с расширениями (pdo_mysql, mbstring, openssl, curl, fileinfo).
- Среда выполнения JavaScript: Node.js версии 20.x LTS (для сборки frontend-активов через Vite).
- Менеджер зависимостей PHP: Composer версии 2.5+.
- Менеджер пакетов Node.js: npm версии 10.x.
- Система контроля версий: Git 2.40+.

6.2 Персональный компьютер (ноутбук)

- 1 Операционная система: Windows 10/11, macOS, Linux.
 - 2 Веб-браузеры: Актуальные версии (не старше двух последних релизов):
- Яндекс.Браузер 24.0+

- Google Chrome 120+
- Microsoft Edge 120+
- 3 Необходимые компоненты браузера:
 - Полная поддержка стандартов HTML5, CSS3 и JavaScript ES6+.
 - Включена поддержка JavaScript.
 - Поддержка Web Storage API (LocalStorage) для хранения токенов авторизации и состояния корзины.
 - Отключены блокировщики скриптов для домена приложения.

6.3 Смартфон (мобильное устройство)

- 1 Операционная система: Android 10+.
- 2 Веб-браузеры: Chrome Mobile, Яндекс.Браузер Mobile.
- 3 Необходимые компоненты браузера:
 - Поддержка HTML5, CSS3, JavaScript ES6+.
 - Включена поддержка JavaScript.
 - Поддержка Touch Events для корректной работы интерактивных элементов.
 - Разрешение экрана от 360px по ширине.

7 Организационно-технические требования

7.1 Этапы разработки

В таблице 1 представлены сроки и этапы разработки интернет-магазина.

Таблица 1 – Сроки и этапы интернет-магазина.

№	Этап	Срок выполнения
1	Предпроектное исследование предметной области	09.02.26
2	Разработка технического задания	19.02.25
3	Проектирование программного обеспечения.	04.03.26
4	Оформление пунктов пояснительной записки	11.03.26
5	Разработка и отладка программного продукта.	13.05.26

Приложение Б ER-модель базы данных

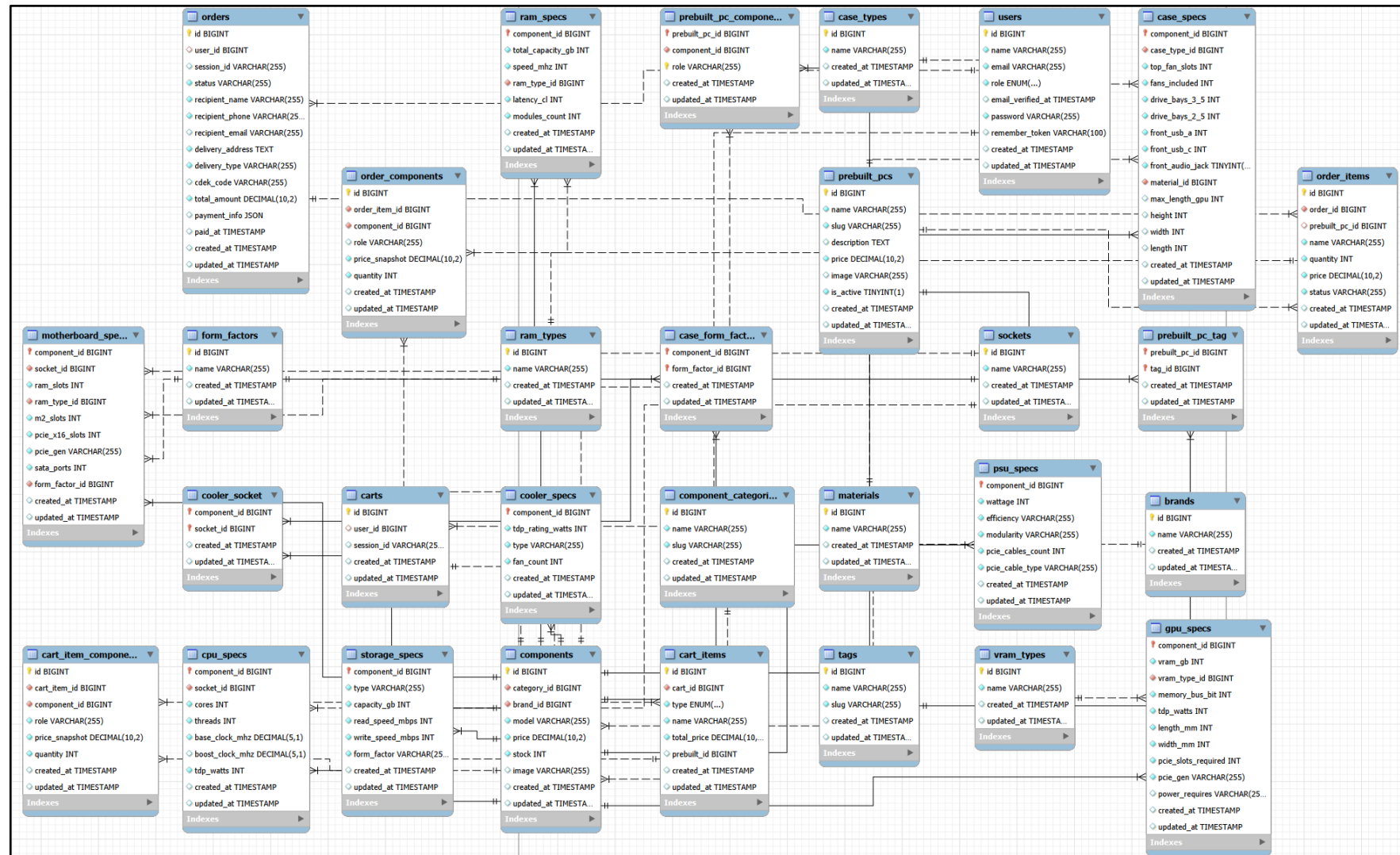


Рисунок – ER-модель базы данных